

VBA 研修テキスト I

入門編

オブジェクトをあやつる

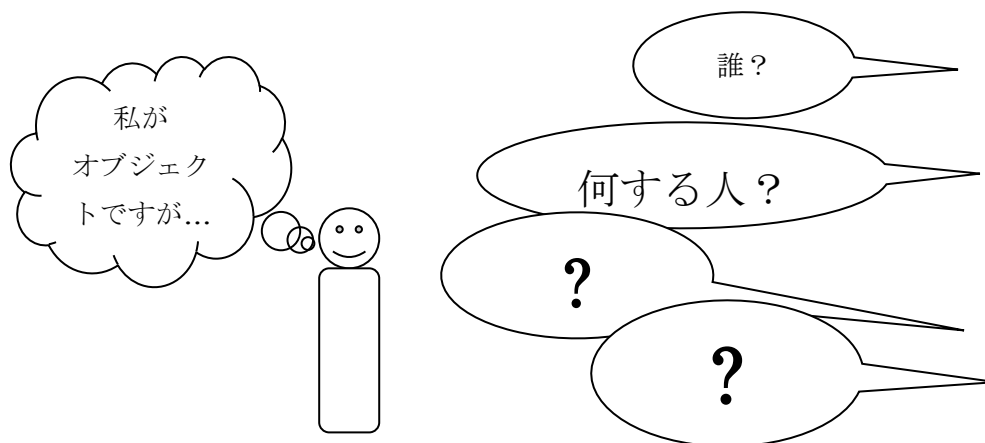
ナビオコンピュータ株式会社

イントロダクション Excel VBA を学ぶ

このテキストの目的は、みなさんにオブジェクトというキャラクターを操る方法を知ってもらい、その面白さを感じてもらうことです。

と言うと、当然、「オブジェクトって、何？」という疑問がわきますね？ それについては、このあとすぐ！

でも、その前に。



さて、仕事で Excel を使っているとき、同じ作業を繰り返しやらなければいけなくなって、うんざりすることがありませんか？ 単にうんざりするだけならまだしも、長時間の作業で効率が落ちてしまったり、あるいは、うっかりミスをやってしまいそうになってヒヤッしたり、さまざまな問題が起きることもあるでしょう。それを防ぐ方法はないのでしょうか？

そもそも、世界最初のコンピュータといわれる ENIAC は、人間が同じ作業（弾道計算）を繰り返し行うのを肩代わりし、かつ、人間よりも素早く正確に実行するために作られたものです。また、コンピュータが普及しはじめた初期の段階において、多くの官庁や企業がコンピュータを導入したのは、徴税業務など、『一定のルールにしたがった計算を多くの人数分繰り返すこと』が必要な仕事あり、その計算処理を人に代わって実行させるためでした。

Excel のようないわゆる『表計算ソフト』は、パソコンが普及し始めた当初から存在し（というより、逆に『表計算ソフトがあったからこそ、ビジネス用パソコンは普及した』といっても過言ではないのですが）、それを使ってきた人たちは、「表計算ソフトもコンピュータ上で動いている以上、ソフト自身に面倒な繰り返し作業を肩代わりさせて、私たちの『うんざり』を減らす方法があるんじゃないか？」と考えていました。あらかじめ作業の手順書を作っておいて、『はい、このとおりにやりなさい』と表計算ソフトに命令したいわけです。

A	B	C	D
ITEM	NO.	UNIT	COST
MUCK RAKE	43	12.95	556.85
BUZZ CUT	15	6.73	101.25
TOE TONER	250	49.95	12487.50
EYE SNUFF	2	4.95	9.90
SUBTOTAL			13155.50
9.75% TAX			1282.66
TOTAL			14438.16

※ 世界最初のパーソナル・コンピュータ用表計算ソフト VisiCalc の画面。まだ VBA のような機能はなかったが、このソフトの登場で PC がビジネスの世界に広がった。

そのためには、手順書を理解し、それに従って動く機能を、ソフトに組み込むことが必要です。そして実際、パソコンの性能が上がるにつれて、多くの表計算ソフトに、そうした機能（「マクロ機能」と呼ばれました）は搭載され、各社はその強力さを競うようになったのです。最初に業界をリードしたのは Lotus 1-2-3 というソフトでした。それを追う Microsoft Excel は、当初、グラフィカルで分かりやすい操作方法だけが売りでした。しかし、やがて 1-2-3 と同等かそれ以上のマクロ機能を搭載し、最終的に表計算ソフトのチャンピオンとして、生き残ることになりました。

Excel VBA とは、Excel に『これこれこういうことをしなさい』と命令するための、作業手順書です。人間が何かをする場合も、ルールにのっとって書かれた手順書があれば、おおむね間違いなく、効率的に作業することができます。ましてコンピュータなら、正確な作業手順書さえあれば、何度でも、ひとつの間違いもなく、しかも高速に作業を行ってくれます。コンピュータ上で動く Excel も同じです。

その作業手順書である Excel VBA には、20 年を超える長い歴史があり（VBA という形なる以前まで含めれば、もっとあります）、その機能は驚くほど発展しています。その気になれば、Windows OS を操作することもできるほどです。そのせいで、この手順書の書き方は、学べば学ぶほど、奥の深いものになりました。したがって、それを学ぶため入口もたくさんあります。

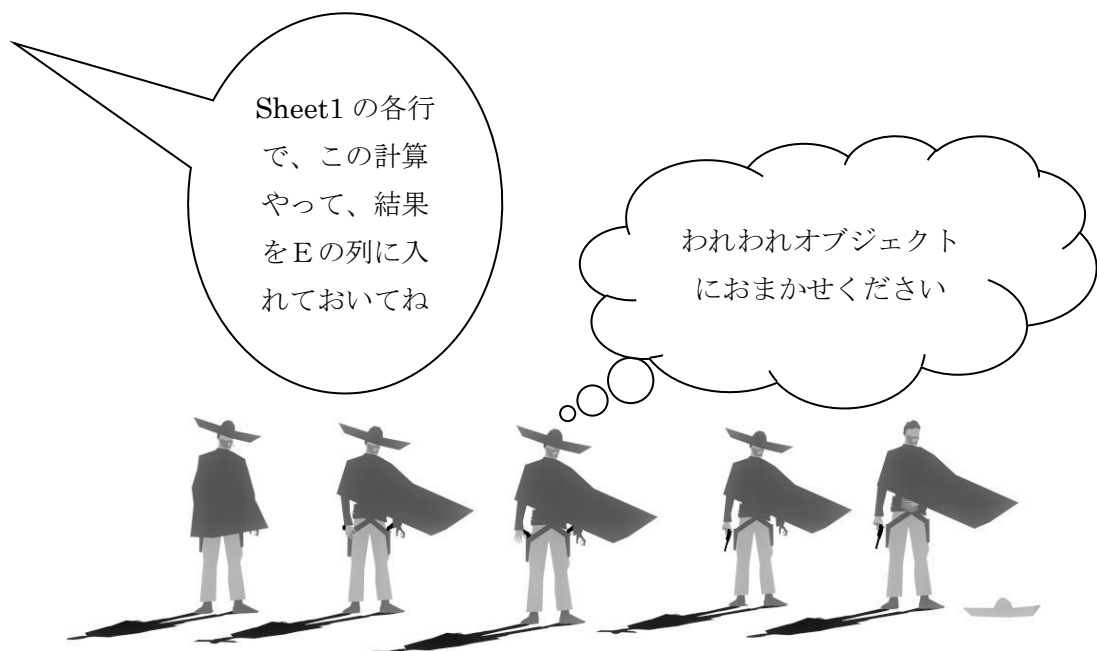
そのたくさんの入り口の中で、このテキストでは、みなさんに、まずオブジェクトというキャラクターを操る方法から伝えてゆこうと思います。

これを学んでもらえば、あとは「オブジェクトに何ができるのか？」「それ以外に Excel VBA にあらかじめ用意されている機能はどんなものなのか？」など、自分であれこれ調べながら、自分の業務に役立ててゆくことができます。ブロックの組み立て方を覚えれば、あとは必要なパーツを入手するだけで、自分の作りたいものを作

ることができるでしょう。VBA の場合、このパーツとは Excel VBA の「オブジェクトやその他の部品の機能」に当たります。そして、それらについては、Web にも、市販本にも、たくさんの情報があります。ですから、まずはブロックを組み立てる面白さを覚えてほしいのです。

さきほど、Excel VBA は高度に発展した、と述べました。しかし、だからといって、入口まで狭き門になったわけではありません。もともとプログラマでない人が使ってきた Excel VBA です。いまでも高度な知識なしに書き、動かすことができます。

まずは、この入門編でオブジェクトを操る面白さを感じ取ってください。



Part.1 一つの命令を書く

このパートのゴール

1. 中身が一行だけの *Sub* プロシージャが書ける
2. オブジェクトとその階層構造を理解し、簡単なセルの操作ができる

I. どんな相手に命令するか

1. オブジェクト

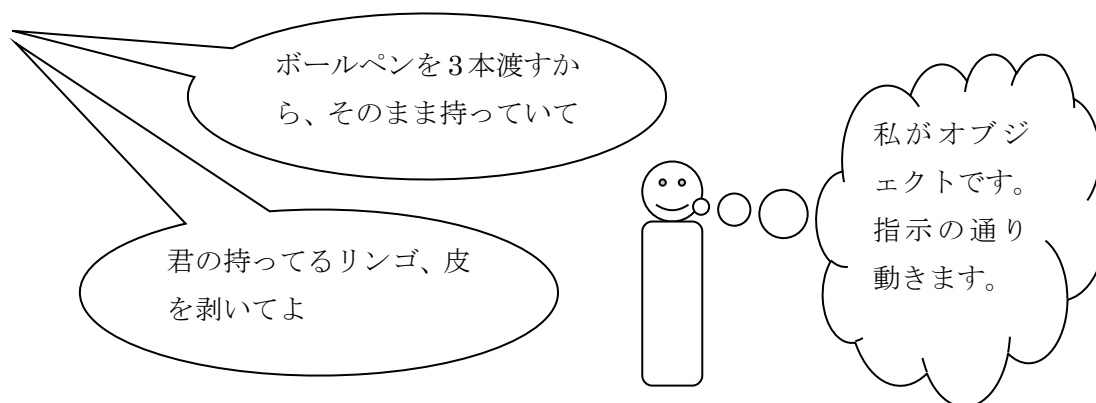
イントロダクションで、VBA では「Excel に『これこれこういうことをしなさい』と命令する」と書きました。しかし、これをさらに具体的に言うと、実は Excel 全体への命令というより

Excel を形作っているいろいろな部品に指示・命令する

ということになります。

人間の組織がさまざまな人から構成されているのと同じように、Excel もさまざまな『もの』から構成されています。そして、「組織に何かをさせる」というのは、実際には「その組織に属する人間に何をさせる」ことであるように、「Excel に作業させる」というのは「Excel を構成する『もの』たちに作業させる」ということです。VBA とは作業手順書ですから、「みんなで協力して、これをやりなさい」とぼんやりした依頼をするのではなく、「A さんにはこれをさせて、B さんにはあれをさせて」と一人ひとりの作業レベルまで、細かく砕いて書く必要があります。

この『もの』のことを VBA では『オブジェクト』と呼んでいます。言葉の響きはちょっと難しそうかもしれませんが、中身は難しくありません。「Excel の中には、たくさんの『オブジェクト』と呼ばれるキャラクターがいて、その連中に言えば、何か仕事をしてくれるらしい」くらいに思ってもらえばいいのです。



というわけで、われわれが「こうしなさい」と指示する相手は、オブジェクトと呼ばれる、Excel のいろいろな部品です。その部品とは、たとえば、あなたが値や計算式を入力するセルであり、そのセルを載せたシートであり、そして、そのシートを編集するためにあなたが開いたブックです。Excel VBA とは、そうした Excel が持つオブジェクトを操作して、さまざまな処理をさせるためのツールなのです。

2. セルというオブジェクト

もう少し、「オブジェクト」について、詳しく説明してみましょう。そして、

そのために、ここではスターウォーズ・エピソード4『新たなる希望』冒頭の R2-D2 にたとえてみます。

【SW EP4 冒頭のストーリー】

レイア姫は、同盟のスパイが入手した『デス・スター』（帝国の巨大要塞）の設計図をロボット R2-D2 に記憶させ、同盟軍の秘密基地へ向かっている。

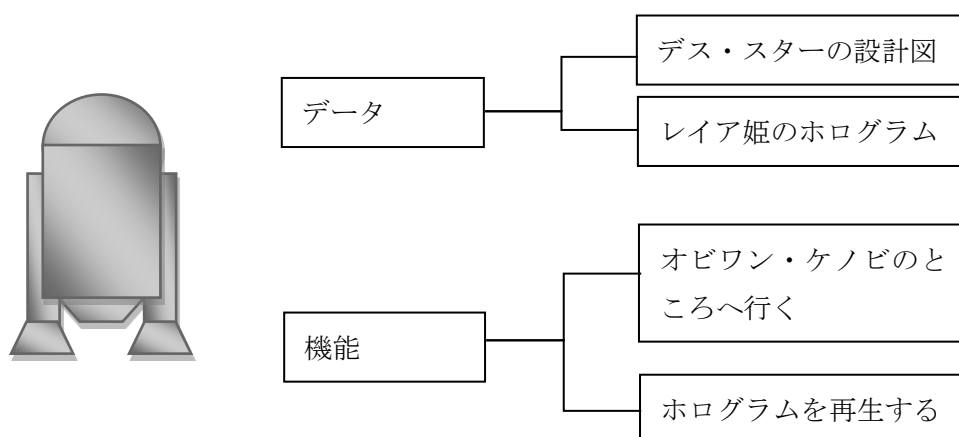
が、その宇宙船をダース・ベェダー率いる帝国軍が急襲する

レイア姫は助けを求めるホログラム(動画)を R2-D2 に託し、タトゥーーン星のオビワン・ケノビに見せるように命令する。

命令を受けて、R2-D2 は C-3PO とともに宇宙船を脱出し、タトゥーーンへ向かうのだが・・・。

さて、ここで R2-D2 は<デス・スターの設計図>と<レイア姫のホログラム>という二つのデータを持っています。つまり、それらを入れるための入れ物が彼の中にあつたということです。

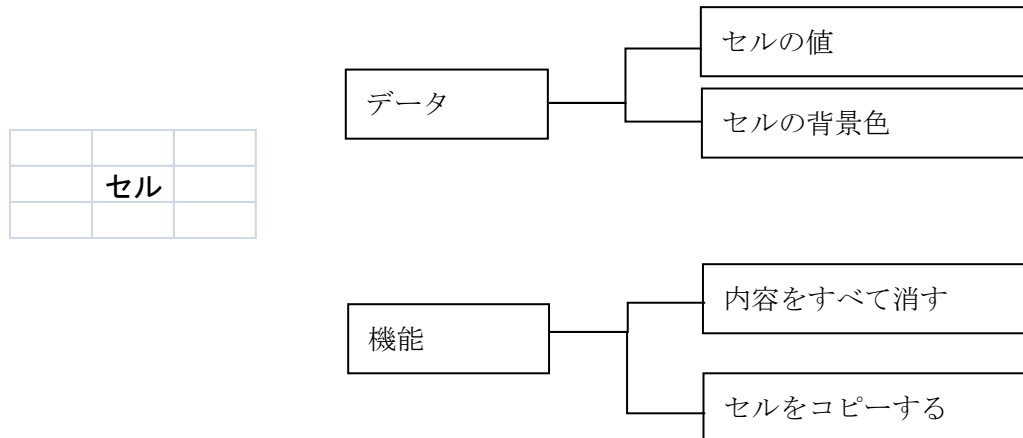
そして同時に、<タトゥーーン星のオビワン・ケノビのところへ行く>、<ホログラムを再生する>という二つの機能も持っていることになります。ロボットですから、元から持っている機能以外の命令を与えられても、実行することはできません。「ボッチェ語の通訳をください」と R2-D2 に言ったら、彼はその機能を搭載した C3P-O を呼んでくるでしょう。自分ではできないからです。



さて、R2-D2 と同じように、Excel のセルというオブジェクトも、データを入れる入れ物と特定の命令を実行する機能を持っています。

たとえばセルに入力された値を保持する入れ物があり、「自分（セル）の背景色はこの色」というように、色のデータを保持する入れ物があります。また入力

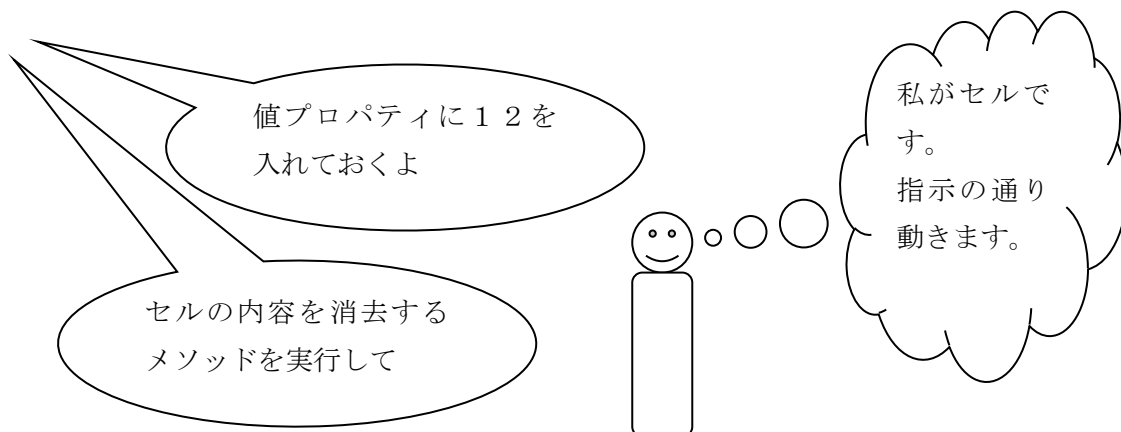
された値や数式をすべてクリアする、自分をどこか別の場所にコピーするなど、
いろいろな命令を実行する機能も、あらかじめ持っています。



オブジェクトの持っている<データ>あるいは<データを入れる入れ物>を
プロパティと言います。

またオブジェクトの持っている<命令を実行する機能>をメソッドと言います。

つまり『オブジェクトを操る』とは、そのオブジェクトが持っているプロパティ
を変更したり、メソッドを呼び出したりして、さまざまな処理を行うことを言
うのです。



では、このことを頭に入れてもらって、次の節で、実際に VBA プログラムを
書いてみましょう。

Excel を起動して、新しい空白のブックを用意してください！

II. 実際に書いてみる

空白のブックは用意できましたか？ それでは始めます。

さて、Excel に対する手順書＝VBA プログラムを書くためには、ちょっとした知識が必要です。それは次のようなものです。

- 何を使って手順書を書くか＝VB エディタ
- どこに手順書を書くか＝標準モジュール
- 手順をまとめる入れ物＝Sub プロシージャ
- 手順を実行するボタン＝実行ボタン

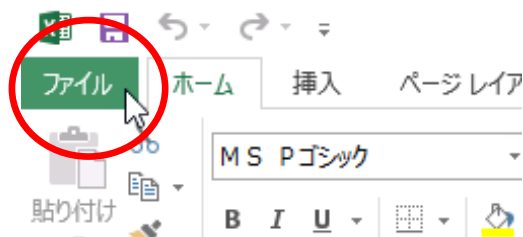
これらを順に説明してゆきます。説明を聞きながら、みなさんも一緒に手を動かしてみてください。

1. 何を使って手順書を書くか＝VB エディタ

VBA プログラムは Excel に付属している VB エディタ (VBE) で書きます。VBE は Excel をふつうにインストールすれば、合わせてインストールされるツールですが、初期の段階では、起動するアイコンが画面に表れません。すこしだけ設定を変える必要があります。

VBE エディタ起動のアイコンを画面にリボンに表示する

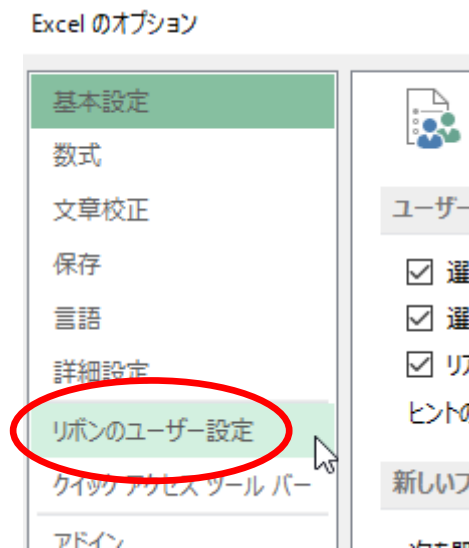
1. まず、Excel 画面左上の、[ファイル]をクリックします。



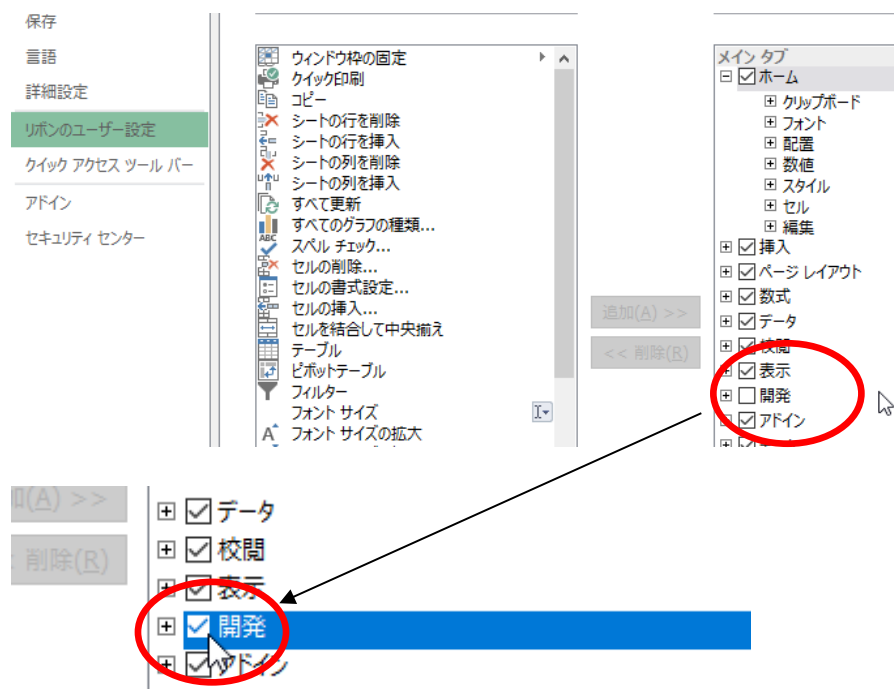
2. 一番下の[オプション]をクリックします。



3. [リボンのユーザー設定]をクリックします。



4. 右の[メインタブ]の[開発]にチェックを入れます（もし、すでにチェックが入っていれば、そのまま[OK]をクリックします）。



5. Excel のメイン画面で、[開発]タブが表示されていることを確認します。



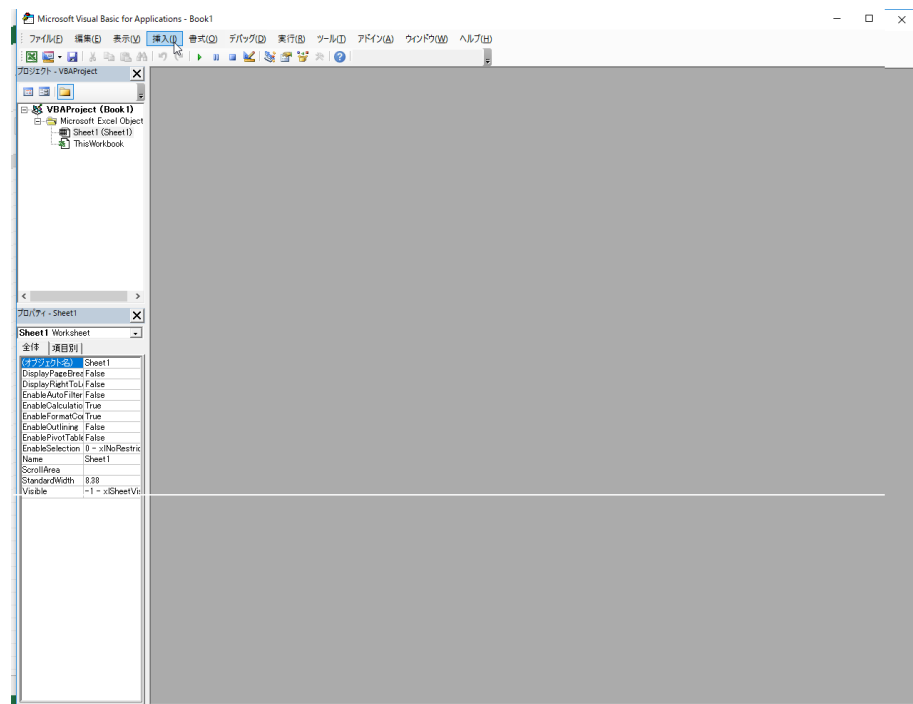
これで設定が終わりました。さっそく、VBE を起動してみましょう。

VBE を起動する

1. [開発]タブをクリックします
2. 右端の[Visual Basic]アイコンをクリックします。



3. VBA エディタが起動します。



なんだか灰色の部分ばかりですね。いったいどこに VBA プログラムを書くのでしょうか？

2. 手順書をどこに書くか？＝標準モジュール

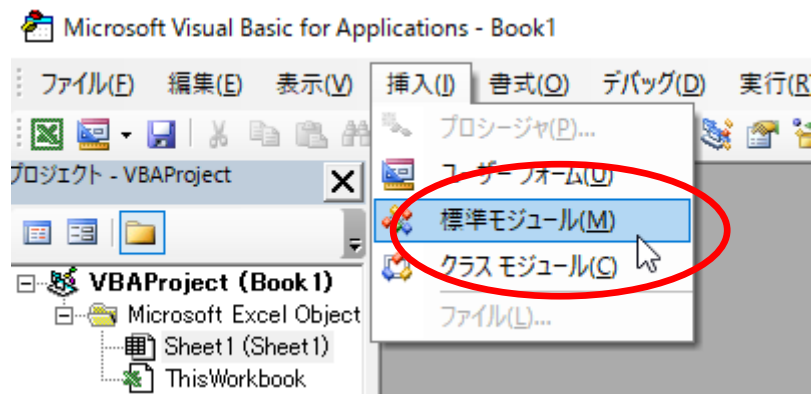
VBA プログラムを書く道具である、VBE を起動するところまで出来ました。でも、まだ VBA プログラムを書く場所がわかりません。

実は、プログラムを書くことができる場所はたくさんあります。しかし、特別な理由がなければ、「標準モジュール」に書くのが普通です。それはまさに VBA プログラムを書くためにある専用のエリアなので、いろいろ使い勝手がよいのです。

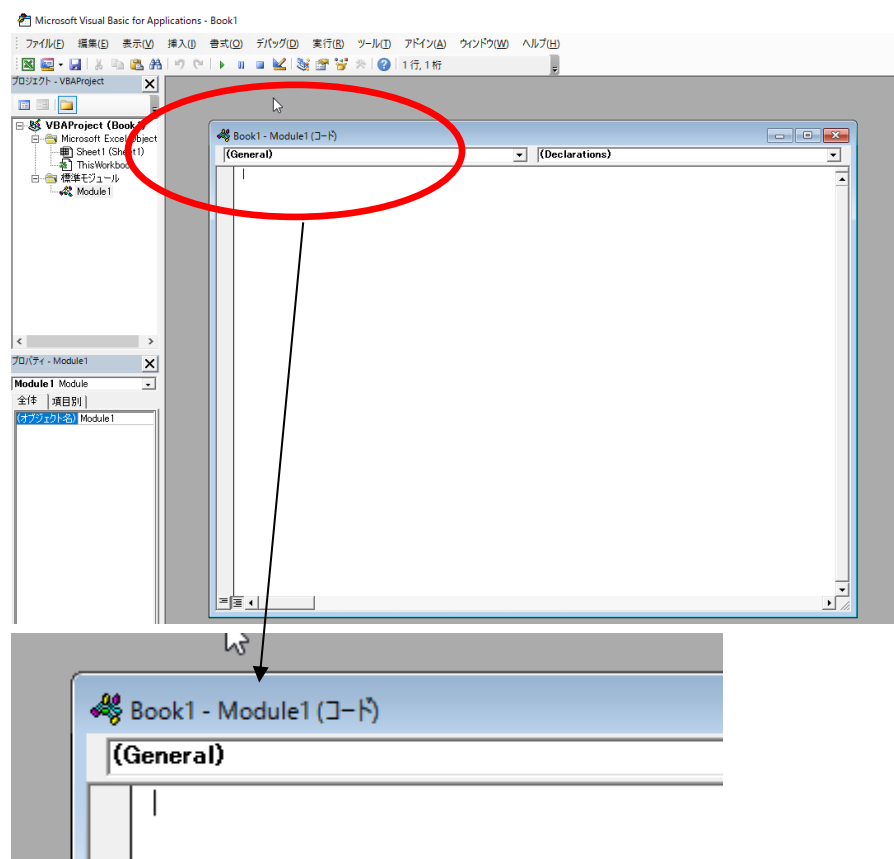
標準モジュールは、ワークシートのように各ブックに所属します。ですが、新しいブックを作った段階では、まだ存在しません。新しいシートを挿入するように、新しい標準モジュールを挿入する必要があります。この操作は VBE 上で行います。

標準モジュールを挿入する

1. VBE のメニューバーの[挿入]をクリックし、表示されるメニューから[標準モジュール]をクリックする。



2. 「Module1」が表示されることを確認する



※ すでに Module1 が存在すれば、自動的に Module2 となります

標準モジュールが画面に表示されました。いよいよこの上に、VBA プログラムを書いてゆきます。まずは、「Sub プロシージャ」という入れ物から。

3. 手順をまとめる入れ物=Sub プロシージャ

VBA プログラムを書くには、最初にまず、「Sub プロシージャ」という入れ物を作ります。

Sub プロシージャとは「ここからここまでが一連の作業です」ということを示すひとつのくくり・まとまりのようなものです。肩ひじ張った言い方では、「VBA プログラムの最小実行単位」となります。でも、これでは何のことかわかりませんね。なので、スポーツに例えましょう。

野球の選手に「ボールを投げなさい」と指示します。そうすると、選手はボールを利き手に握り、腕をいったん後ろに回して、ふたたび前に振り出し、適当なところで握っていたボールを放す、という一連の手順を実行します。一連の手順が「ボールを投げなさい」という一つの命令にまとめられているわけです。

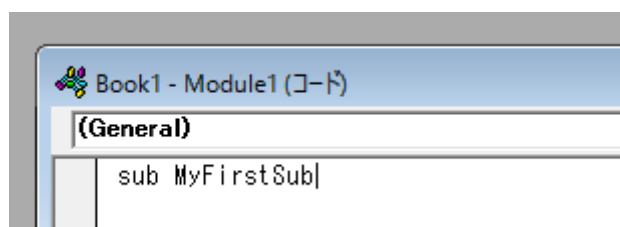


© freedesignfile.com

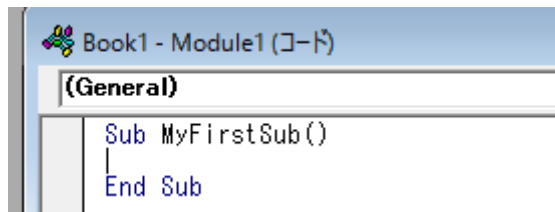
Sub プロシージャも同様です。たとえば、「HaisouryoKeisan」という Sub プロシージャに商品の配送料金を計算するための一連の手順を書いたとします。そうしておくことで、ユーザーは「HaisouryoKeisan を実行しなさい」と Excel に伝えるだけで、配送料を計算させることができます。そんなふうに一連の手順をまとめるための入れ物が、Sub プロシージャなのです。

では、Sub プロシージャを書いてみましょう！

標準モジュールに、次のように入力してください。あ、でも、書き終わっても、まだ Enter キーはたたかないで！



MyFirstSub の後ろでカーソルが点滅していますね？ さあ Enter キーをたたいてみましょう。



1回キーをたたいただけなのに、たくさんの文字が挿入されました。それに”sub”と小文字で書いたのに、自動的に”Sub”と頭が大文字になりました。

VBE には、スマホの予測変換のように、必要な文字を予測しながら、自動的に書き込んでくれる、便利な機能があります。すでに書き込んだ文字も、必要に応じて変更してくれます。そのせいで、最小限のキー入力でプログラムを書くことができるのです。

ところで、今、あなたが標準モジュールに書き込んだ（そして VBE が補足してくれた）文には、どんな意味があるのでしょうか？

まず、”Sub”は「ここから先は Sub プロシージャの記述ですよ」という宣言です。必ず、”End Sub”というキーワードと一緒に使います。”Sub”と”End Sub”に挟まれたエリアが、ひとつの Sub プロシージャであることを示します。

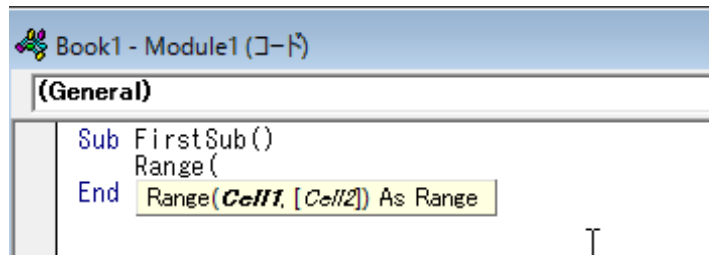
次の”MyFirstSub”が Sub プロシージャの名前です。Excel から呼び出すときは、この名前呼び出します（呼び出し方は後ほど）。うしろに空っぽのカッコ”0”がくっついていますが、これについては、ここでは説明しません。今はまだ、「Sub プロシージャって名前の後ろに”0”が付くんだ」と単純に覚えておいてください。それで十分です。

さて、入れ物ができましたが、まだ中身がありません。この入れ物に中身を入れましょう。

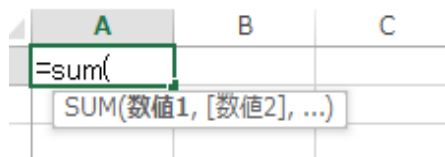
下のように” Range(“A1”).Value = 1000”と入力してください。これはあるオブジェクトへの指示です。入れ物の部分と中身の部分をパッと見で区別できるように、行の頭で TAB キーを打って、中身を入れ物より TAB ひとつ分、右になるようにして、書きはじめましょう。

```
Sub MyFirstSub()  
    Range(“A1”).Value = 1000  
End Sub
```

あれ？ でも、書いている途中で、下に何か出てきましたね。



Excel で作業をしているとき、これによく似たものが出てきませんでしたか？
たとえばセルに数式を入れるとき…



そう、こんなやつです。VBE もセルと同じように、自動的に「こんなふうにかきなさい」と書式を教えてくれるのです。でも、残念ながら英語表記…。

気を取り直して、続きを書いてみましょう。

今度は、“Range(“A1”).”とドットまで書いたところで、ドロップダウンリストが出てきました。



これはセルが持っているプロパティやメソッドのリストです(※)。  のマ

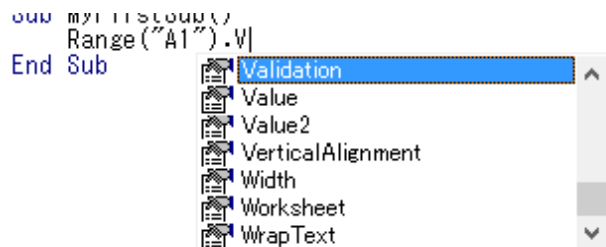
ークがメソッドで、  のマークがプロパティです。

※ 本当は、Range オブジェクトのプロパティやメソッドです。Range とはセル範囲を示すもので、単一のセルを指す場合もありますが、本来の意味は、複数のセルを含む一定の領域を示すものです。ただ、しばらくの間、説明を簡単にするために、複数のセルの場合を無視します。Range=セルと思って、説明を読んでください。

このリストからマウスで“Value”プロパティを選ぶこともできます。でも今は(キ

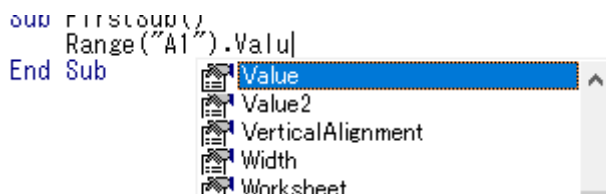
一ボードから手を放してマウスに持ち替えるのも面倒ですから)、そのままキー入力続けましょう。

“V”と打つと、表示が少し変わりました。



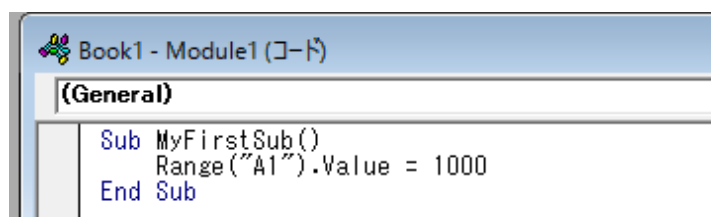
“V”で始まるプロパティにリスト表示が飛びました。続けて打ちます。

“Valu”まで入力すると、リスト上の Value が青くハイライトされましたね。



ここで TAB キーを押すと、“Value”と確定されます。もちろん、最後の“e”を自分で打ってもかまいません。

Value というプロパティ名まで書けたら、残りを入力してしまいましょう。



書き終えましたか？ 素晴らしい。では、プログラムを実行する前に、あなたが書いた『中身』の意味を、簡単に説明します。

”Range(“A1”)は A1 セルを指します。”Value”はセルに入っている値を示すプロパティ名です。A1 セルの値プロパティを示す場合は、

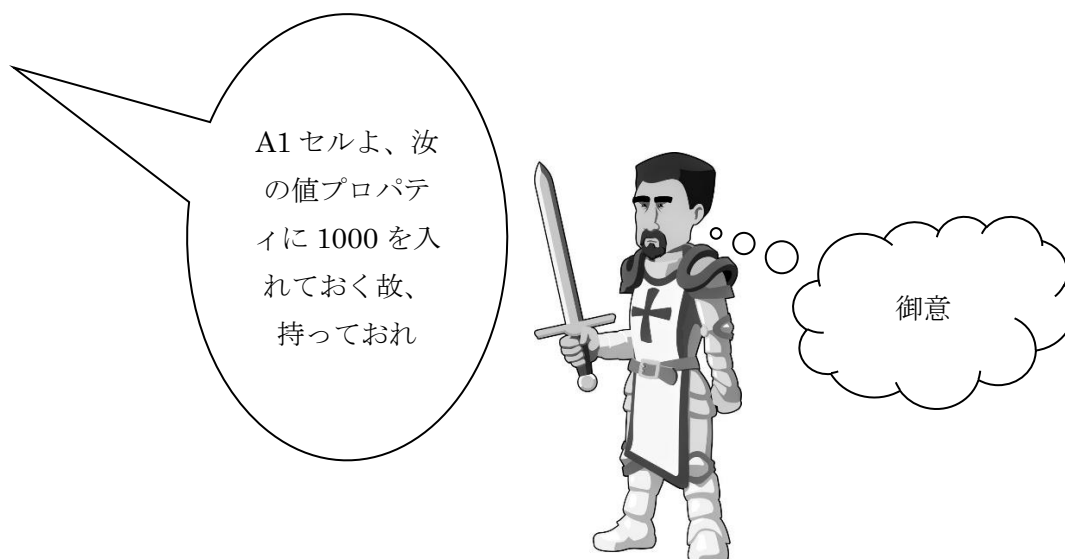
Range(“A1”).Value

というふうに、オブジェクト名とプロパティ名を”.”でつながります。

そして、“=”は代入演算子とよばれるものです。ここでは”1000”を A1 セルの値プロパティに代入しています（『代入』という数学用語にアレルギーをお持ちの方は、「”1000”を A1 セルの値プロパティに入れています」と読み替えてください）。詳しくは、後ほど説明しますが、今は「プロパティに何かを入れるときは、“=”という記号を使うんだ」くらいに思っていてください。

というわけで、この一文は「A1 セルさん、君の値プロパティに 1000 を入れる

から、そのまま持っていてね」というオブジェクトへの作業指示になるのです。

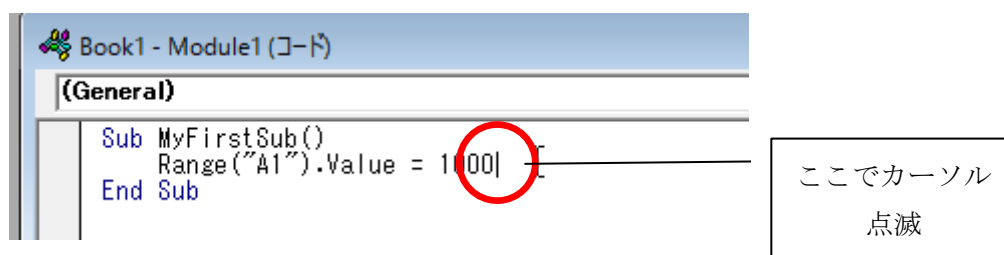


4. 命令を実行する=実行ボタンとダイアログ

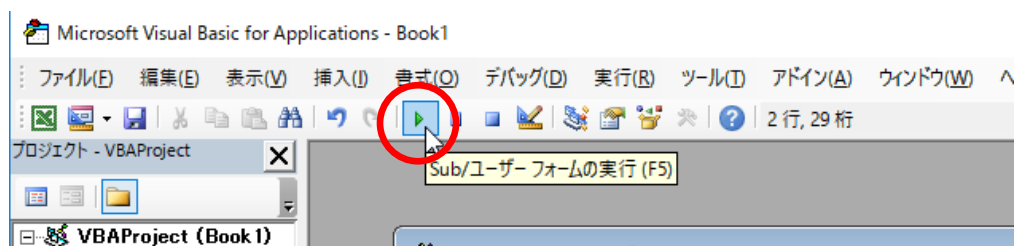
では、実行しましょう。方法はいくつかありますが、ここでは二つの方法を紹介します。

まずは、VBE から実行する方法です。

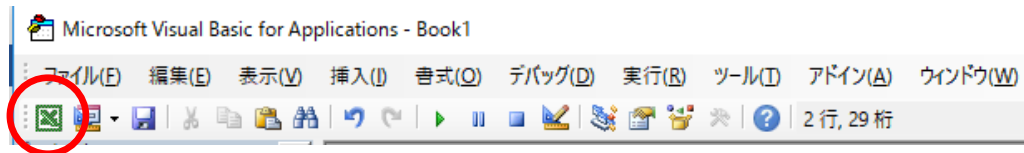
実行したい Sub プロシージャ内のどこか（ここでは Sub MyFirstSub()~End Sub の間のどこか）をクリックして、そこでカーソルが点滅するのを確認します。



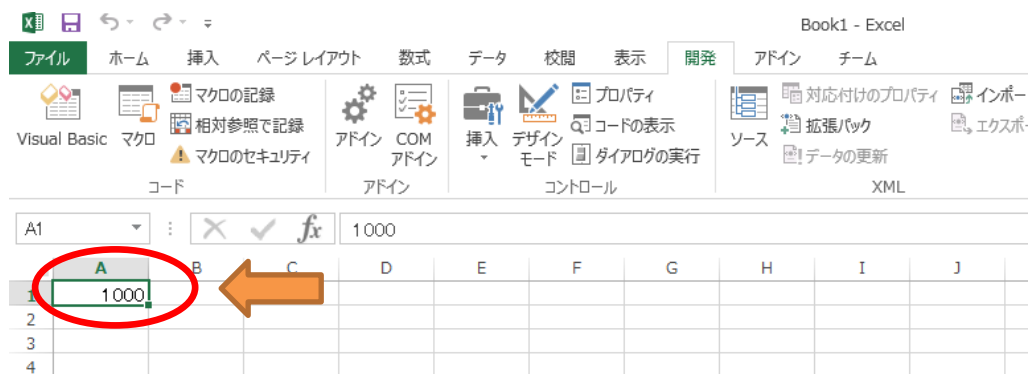
エディタ上の[実行ボタン]  をクリックします。



何も変わらない？　そうですね。この画面では、何が起こったのかわかりません。
なので、エディタ上の左端の Excel アイコンをクリックして、Excel の画面に切り替えます



A1 のセルに 1000 という値が入っているでしょうか？

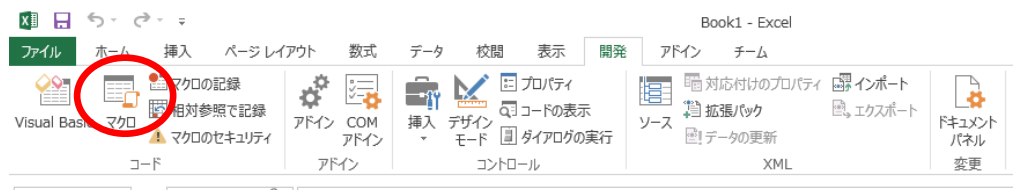


入っていれば、OK！ あなたの書いた指示に従って、A1 セルが仕事をしてくれました！ 簡単な仕事ですが、これが第一歩です。オブジェクトたちに対する指示のスキルが上がるにつれて、もっともっと複雑な仕事もしてくれます。気長に学んでゆきましょう。

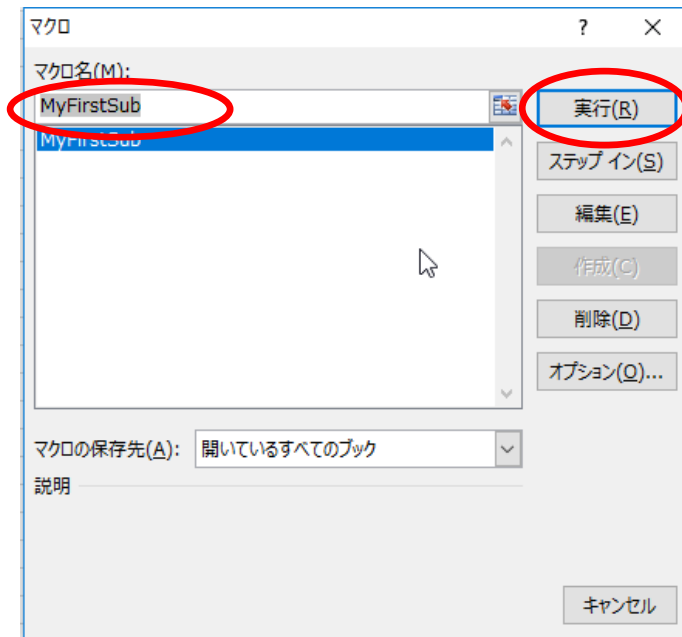
でもプログラムを実行させるのに、毎回わざわざ VBE を立ち上げなければならないのでしょうか？　いえいえ、そんなことはありません。今は書いたばかりですから VBE を開いていますが、ふだんは Excel 画面しか見ませんよね。

なので、今度は Excel から実行してみます。実行できたことを確認するために、一度、A1 に入った 1000 の値を消してください。消し終わったら、次のように”MyFirstSub”を呼び出して実行しましょう。

Excel のリボンの[開発]タブを開き、左から 2 番目の[マクロ]をクリックします。



マクロの実行ダイアログが表示されます。



今はひとつだけですが、ここには標準モジュールなどに書かれている Sub プロシージャの名前がすべてリストアップされます。

では、MyFirstSub が選択されているのを確認して、[実行]をクリックしましょう。

先ほどと同じように、A1 セルに 1000 が入るはずです。

＊

『VBA プログラムを書いて、それを実行する』という流れは、なんとなくつかめたでしょうか？

ここで復習のために、練習問題をやってみましょう。もう一度 VBE を開いて、Module1 の” End Sub” の下を書いてゆきます。ただし、3 行ほど空けましょう。二つの Sub プロシージャがくっついていると、見た目がごちゃごちゃして、汚くなりますから。

【練習問題 1】

D3 セルに 787 という値を入れる Sub プロシージャを Exercise01 という名前で書き、マクロの実行ダイアログから実行しなさい。

＊

・・・うまくいきましたか？ OK？

おめでとうございます！

あなたは人生で最初（？）の VBA プログラムを書きあげ、

そして動かしました！



Pachi! Pachi!

では、次のステップへ進みましょうか。

Ⅲ. オブジェクトの呼び名

前章は、『へい！ とにかく VBA プログラムを書いて、動かしてみようぜ！』というノリで進めました。その中で、セルのプロパティについて簡単な説明をしましたが、まったくピンポイントだけの説明でした。

なので、この章で、もう少し「セルの操作方法」の知識を広げましょう。セルに文字を入れる方法や、シートが二つ以上ある場合の操作方法を説明してゆきます。

でも、まずは前章の復習をちょっとだけ。

1. セルのプロパティを操作する

セルの指定方法について、あらためて述べます。

Excel には、たくさんのオブジェクトがあります。したがって、ただ、「これをこうしてください」ではなく、「XX さん、これをこうしてください」というふうに、作業すべき人が「XX さん」であることを、はっきりと示さなければなりません。VBA 上でオブジェクトを特定するには、以下のような書式を使います。

オブジェクトの種類(“名前”)

【例】

A1 セル → Range(“A1”)

Sheet1 シート → WorkSheets(“Sheet1”)

また、それぞれのオブジェクトには複数のプロパティがあるので、操作対象のプロパティを、

オブジェクトの種類(“名前”).プロパティ名

というふうに記述して、指定します。

【例】

A1 セルの値プロパティ → Range(“A1”).Value

Sheet1 シートの名前プロパティ → WorkSheets(“Sheet1”).Name

【例題】

A2 セルに数値 909 を入れるプログラムを、Example01 という名前で書きさい

【解答】

```
Sub Example01()
```

```
    Range(“A2”).Value = 909
```

```
End Sub
```

前章の復習ですから、簡単ですね？

とりあえず、この通りに書いて、実行してみてください。方法は、前節で説明した二つのうちの、どちらを使ってもかまいません。

次は数値ではなく、文字列を入れるプログラムです。

文字が二つ以上並んだものを、プログラミングの世界では「文字列」といいます。というか、実は文字が一つだけのものも、別にするのがめんどくさいので(?)、「文字列」として扱うことがほとんどです。

「文字列」という言葉は、Excel の書式でも使いますね。あれと同じです。数値であれば、足し算・引き算などの数値計算ができますが、文字列ではできません。文字列は、原則、人に読ませるためのものです。

下の例題はセルに文字列を入れる例です。ここでは” ロースかつ丼” という文字列を” (ダブル・クォート)で囲んでいることに注意してください。そうしないとエラーになります。

【例題】

A1 セルに、” ロースかつ丼” という文字列を入れるプログラムを、Example02 という名前で書きなさい

【解答】

```
Sub Example020
    Range(“A1”).Value = “ロースかつ丼”
End Sub
```

これも、この通りに書いて、実行してみてください。「1000」となっていたセル A1 が” ロースかつ丼” に変わるはずです。

2. セルのメソッドを操作する

今までプロパティばかりに触ってきたので、今度はメソッドを呼び出してみましょう。書式はプロパティのときと同じように

オブジェクトの種類(“名前”).メソッド名

というふうに、記述します。

【例】

A1 セルの数式や文字を削除する → Range(“A1”).ClearContents

A1 セルをコピーする → Range(“A1”).Copy

【例題】

A1 セルの数式や文字を削除する Sub プロシージャを、Example04 という名前で書きさい

【解答】

```
Sub Example04()  
    Range( "A1" ).ClearContents  
End Sub
```

例によって、これも、この通りに書いて、実行してみてください。「ロースかつ丼」の文字列が消えたでしょう？

＊

ではここで練習問題です。

【練習問題 2】

E3 セルに「比内地鶏親子丼」という値を入れる Sub プロシージャを Exercise02 という名前で書きなさい

【練習問題 3】

E3 セルの文字列を削除する Sub プロシージャを Exercise03 という名前で書きなさい

＊

練習問題はうまくできたでしょうか？

これまでのところ、セルの特定のプロパティやメソッドだけを使用しています。実際には Excel にはいろんなオブジェクトがあり、セルにもいろんなプロパティやメソッドがあります (” Range(“A1”).” と打ったとき、ドロップダウンリストにいっぱい表示されましたよね？)。ただ、いきなりそれらの説明を始めても、みなさんの頭を混乱させるだけだと思います。まずはシンプルなプロパティ・メソッドだけを使って、VBA の基本を理解しましょう。

3. オブジェクトの階層構造

Excel にはたくさんのオブジェクトがあります。あまりにも多い (セルだけを考えても、一つのシートにどれだけあるのでしょうか？ そして、多くの場合、一つのブックには複数のシートがあるのです) ので、これらすべてに、他とかならない名前を付けるは大変です。つまり、別のオブジェクトに同じ名前をつ

けてしまいがちなのです。

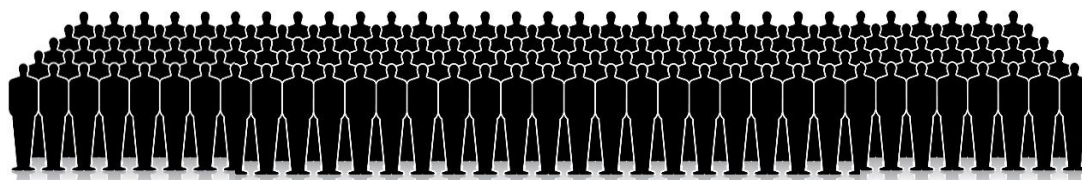
しかし、VBA のなかで、どのオブジェクトに対する命令なのかをあいまいにすることは、とても危険です。VBA は他のプログラム言語と違って、ある程度のあいまいさを許容してくれるのですが、しかし、まちがったオブジェクトに指示が届けば、まちがった処理が行われて、とんでもない結果になりかねません。

そういった事態を避けるために、階層構造を使った命名が行われています。「階層構造」といっても、別に難しいものではありません。たとえば（またまたスター・ウォーズにたとえれば）、こんな感じです。

反乱同盟軍＞デス・スター攻撃部隊＞X-ウィング・レッド中隊＞ウェッジ

反乱同盟軍にウェッジという名前のパイロットは何人もいるかもしれませんが。でも、「デス・スター攻撃部隊」の中の、「X-ウィング・レッド中隊」にいる「ウェッジ」といえば一人だけでしょう。

反乱同盟軍



の中の



ウェッジさん（いっぱいいる！）

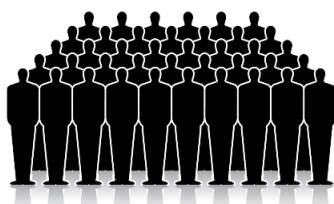
でも、こんなふうに絞り込めば、



反乱同盟軍



の中の



デス・スター攻撃部隊



の中の



X-ウィング・レッド中隊



の



ウェッジさん

は一人だけ

同様に A1 というセルはいくつもあるかもしれませんが、book1 というブックの sheet1 というシートの A1 セルは、そのとき動いている Excel にとって、たった一つだけしかありません（同じ名前のブックは、たとえ別のフォルダにあるものでも開けないのは、知っていますね？）。

この階層構造の中の親子関係は、”.” でつなぐことで表します。

Book1 の Sheet1 の A1 セルであれば、下記のようになります。

```
Workbooks("Book1").Worksheets("Sheet1").Range("A1")
```

したがって、Book1 の Sheet1 の A1 セルの値プロパティは、

```
Workbooks("Book1").Worksheets("Sheet1").Range("A1").Value
```

と記述します。

＊

では、上の記述方法を踏まえて、いくつか練習問題を解いてみてください。ただし、Sheet2 がないと、実行するときにエラーになってしまうので、一度 Excel 画面に戻って、Sheet2 を挿入しておきましょう。

【練習問題 4】

Sheet2 の A1 セルに 813 という値を入れる Sub プロシージャを Exercise04 という名前で書きなさい

【練習問題 5】

Sheet2 の B2 セルに” 米沢牛ステーキ丼 ” という文字列を入れる Sub プロシージャを Exercise05 という名前で書きなさい

【練習問題 6】

同様に、Sheet1 または Sheet2 の好きなセルに、好きな文字列を入れる Sub プロシージャを Exercise06 という名前で書きなさい

＊

練習問題はどうでしたか？ うまく動いたでしょうか？

では、このパートの最後に、今まで使ってきたブックを保存しましょう。

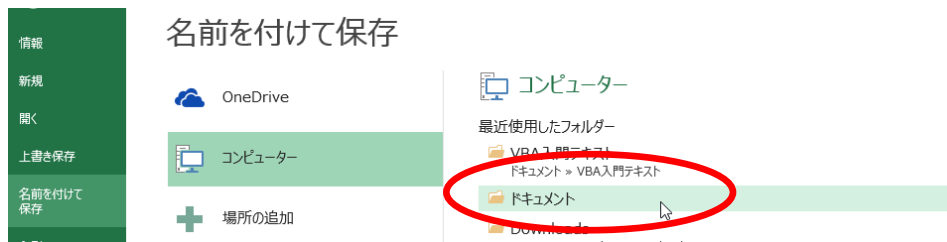
まず、普通の上書き保存にトライしてみます。

ファイルメニューをクリックして

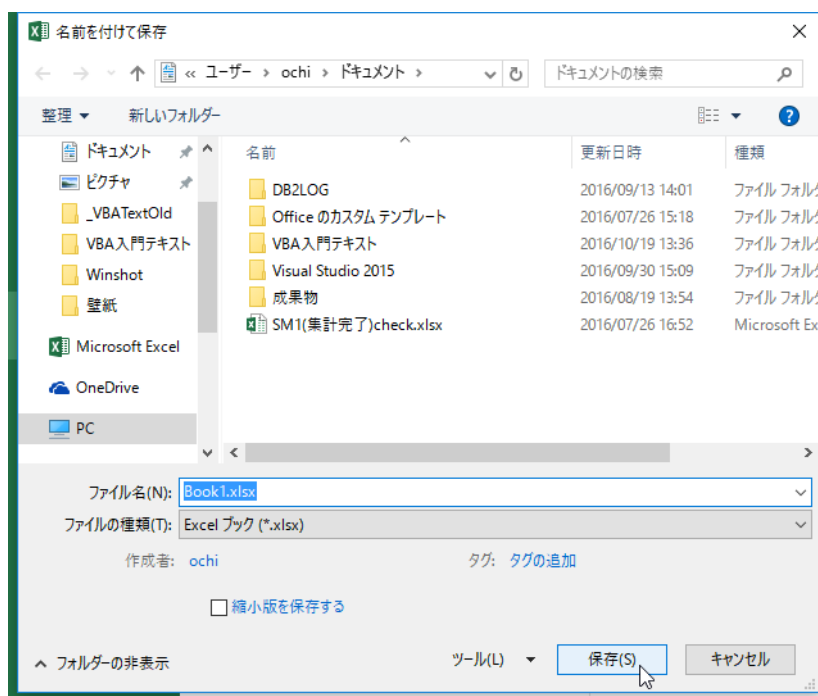


上書き保存をクリック。

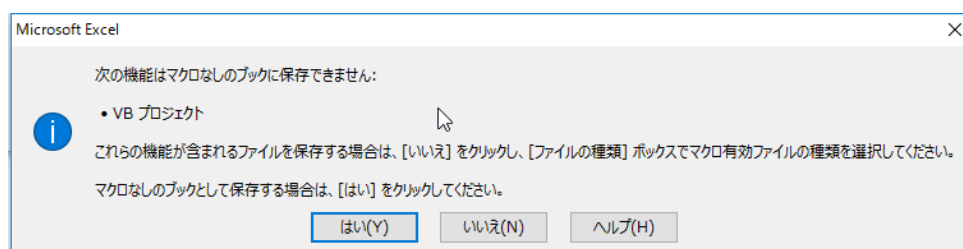




まだデフォルトの book1 のままだったので、「名前を付けて保存」のダイアログがでました。なので、保存先のフォルダを選択して、



保存ボタンをクリックします。あれ？



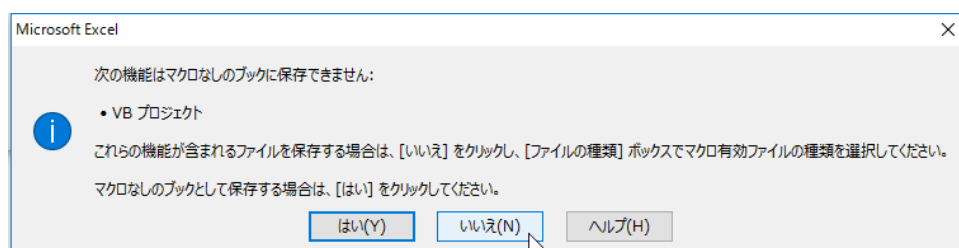
なんか怒られました。なぜ？

これはセキュリティ上の理由で、VBA プログラムを含んだブックは、そうでないブックと同じ拡張子(.xlsx)では保存できず、別の拡張子(.xlsm)にしなければならないからです。

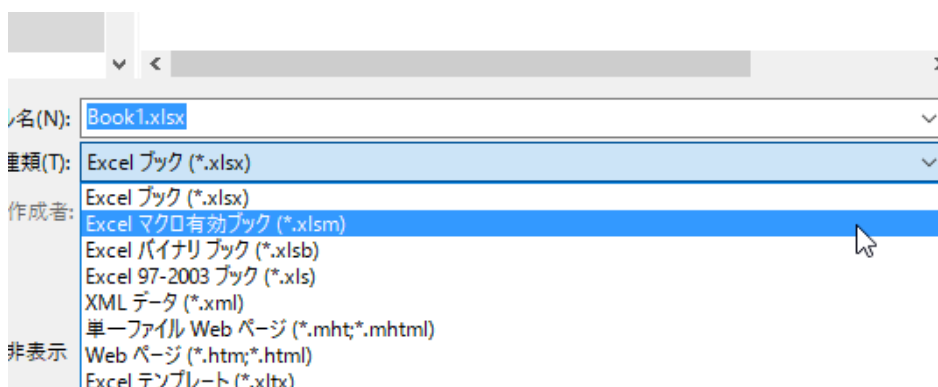
以前は同じ拡張子(.xls)でした。そのため、Excel ファイルに悪意のある VBA プログラムを仕込んで、しかも、ファイルを開くと同時に自動実行される仕組みにして、メールで送りつけるクラッカーがいても、実際に開いてみるまで分かりませんでした。結果、うっかりファイルを開いてパソコンをウィルスに感染させてしまった事故が、全世界で発生してしまったのです。

その対策として、マイクロソフトは「VBA プログラムなし」と「VBA プログラムあり」の Excel ファイルを、拡張子を見ただけで区別できるようにしました。

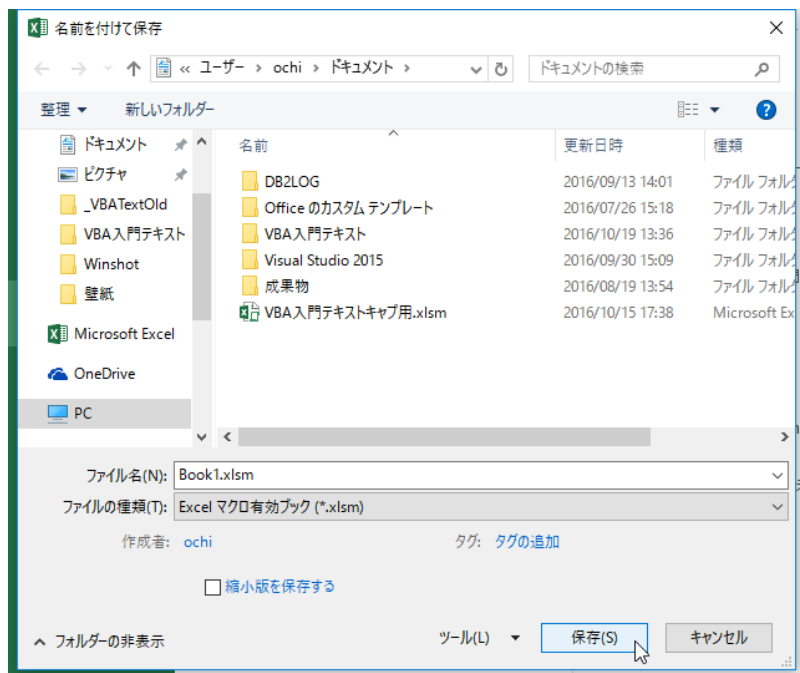
では、拡張子.xlsm で保存しましょう。



まず、上のダイアログで「いいえ」を選択します。



そして、「種類」の項目で、Excel マクロ有効ブック (*.xlsm)を選びます。



「名前を付けて保存」ダイアログが出てくるので、フォルダを選択し、必要であればファイル名を変更し、保存ボタンをクリックします。

今度は普通に保存できました。

さて、これでこのパートは終わりです。少しはVBA とに VBE に慣れましたか？
慣れた。それはよかった。

Good Job!



ここまでは一つのオブジェクトに一つの指示を一回出すだけでした。次のパートでは、まず Excel の操作を VBA に記録する方法を学び、そこに記録されたプログラムから、複数のオブジェクトに複数の指示を出す方法を学びます。

どんどん、プログラムらしくなってきますよ。期待してください。

Part.2 複数の命令を順番に実行する

このパートのゴール

1. 「マクロの記録」操作ができる
2. 順次実行を理解する
3. ステップ実行の方法を知る

I. マクロの記録

今まで、VBE で簡単なプログラムを書く作業をやってもらいました。

でも、実は Excel の操作を VBA プログラムとして記録してくれる、便利な機能があります…と言ったら、怒りますか？

「だったら、VBA の書き方を覚える意味なんて、ないじゃん！」

いえ、覚える意味はあります。この便利な機能（「マクロの記録」と言います。知ってる人、いますよね？）には、致命的な問題があるからです。どんな欠点か？それはまた、後ほど。

VBA を学ぶ人にとって、この機能のいいところは、オブジェクトが働いているさまを、VBA プログラムの形で見られることでしょう。映画やドラマの業界で、出来上がった映像から逆にシナリオを起こす仕事があるそうですが、ちょうどそんな感じです。オブジェクトたちの行動履歴が、ドラマ脚本の俳優の動きのように記述されているのを、読むことができます。

では、「マクロの記録」方法から、説明してゆきましょう。

マクロの記録の方法

1. リボンの[開発]タブを開き、[マクロの記録]をクリック



2. ダイアログが出てくるので、マクロ名を確認して（必要であれば、ここで変更する）、OK をクリック

マクロの記録

マクロ名(M):
Macro1

ショートカット キー(K):
Ctrl+

マクロの保存先(I):
作業中のブック

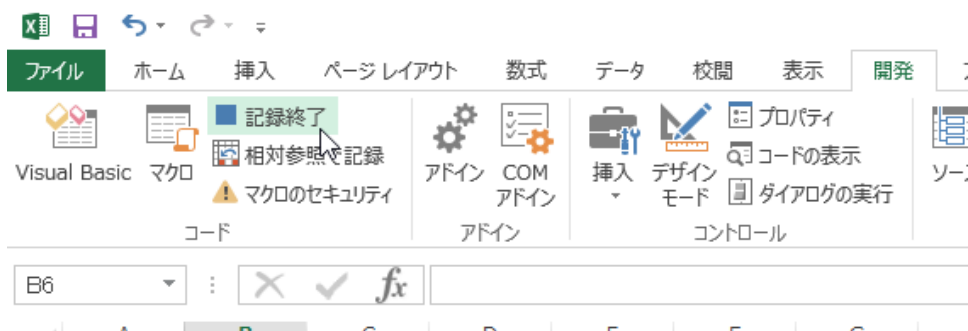
説明(D):

OK キャンセル

3. Excel 上で操作を行う（ここでは、4 つのセルに値を入れ、その下に数式を入力しました）

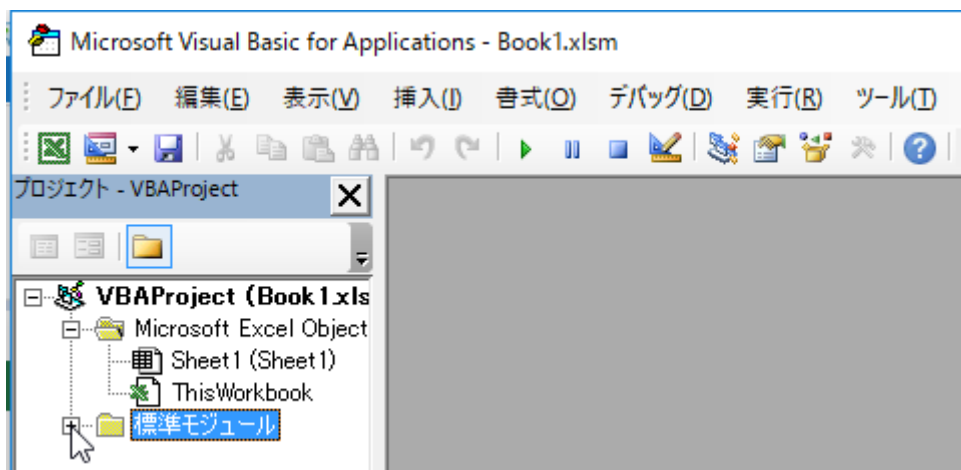
	A	B	C	D
1				
2		ここに値+入れる		
3		その下にも値を入れる		
4		1999		
5		5		
6		=B4*B5		
7				

4. 記録終了をクリック

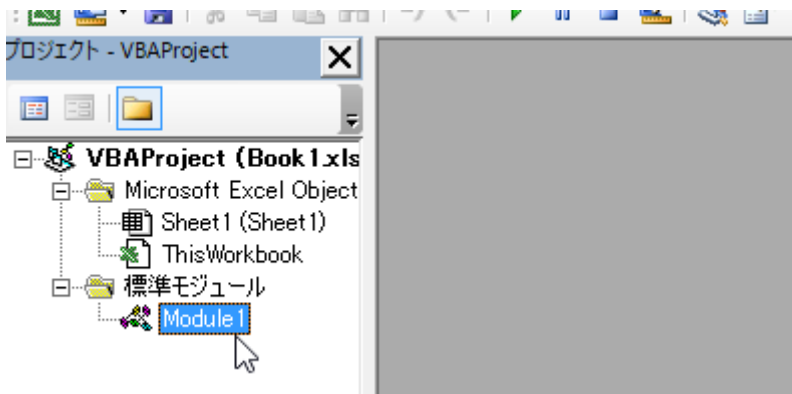


それでは、記録した中身を見てみましょう。

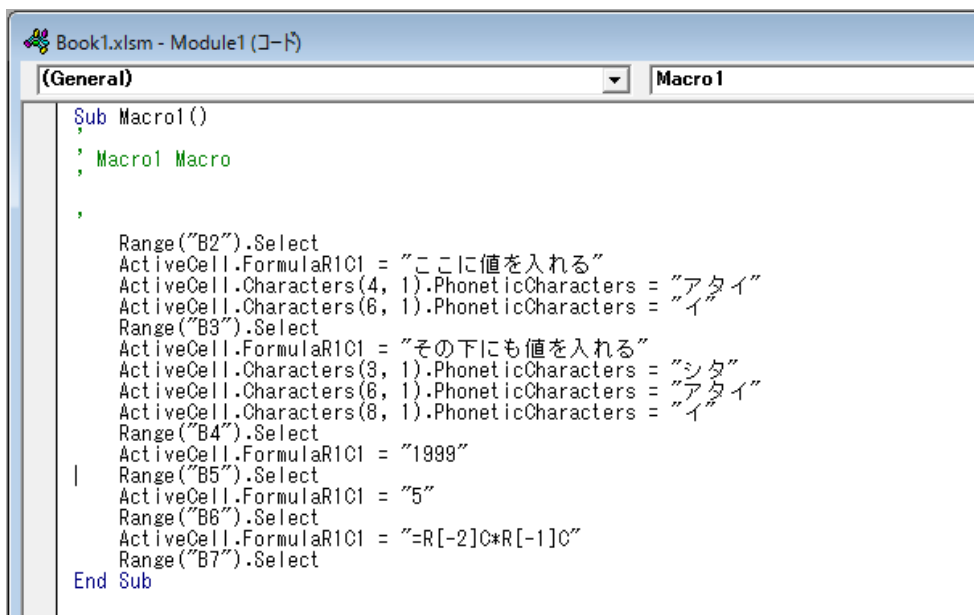
5. VBE を開いて、左のツリーから、[標準モジュール]の前の[+]をクリックして、フォルダを開きます。



6. 中の Module1 をダブルクリックで開きます。



7. はい、さっき記録したマクロが表示されます。



前のパートで書いたプログラムと、似ていると感じましたか？ それとも全

然違うと思いませんか？

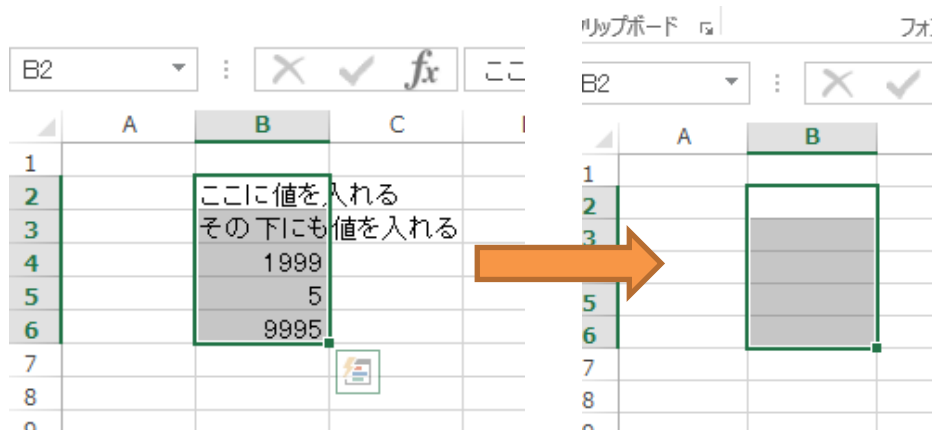
ここには Excel のさまざまなオブジェクトの動きが、記録されています。Excel を操作するということは、実際には Excel のオブジェクトに働いてもらっていることだ、と実感できるのではないのでしょうか？ 後ほど、どのオブジェクトがどういう仕事をしているのかを、一行一行、解説します。

でも、その前に、記録したマクロがちゃんと動くのか、実行して確認しましょう。

1. Excel 表示ボタンをクリックして、いったん Excel の画面に戻ります。



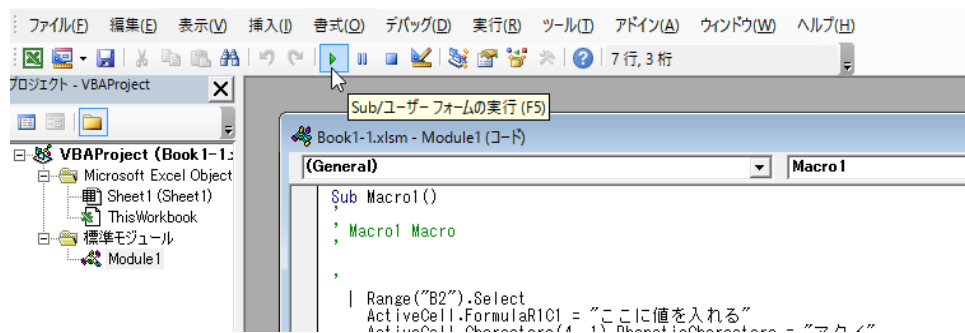
2. 後で実行したことが分かるように、画面をクリアします。



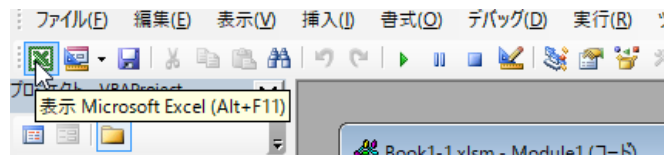
3. [Visual Basic]アイコンをクリックして、再び VBE を開きます。



4. Sub プロシージャ Macro1 の中にカーソルを置いて、実行ボタンをクリックします。



5. Excel 表示ボタンをクリックして、Excel の画面に戻ります。



6. セル B2 から B6 にそれぞれの値・数式が入っていることを確認します。

	A	B	C	D
1				
2		ここに値を入れる		
3		その下にも値を入れる		
4		1999		
5		5		
6		9995		
7				

はい、ちゃんと実行されました。でも、一気にやられると、どのオブジェクトがどういう仕事をしているのか、分かりませんね。

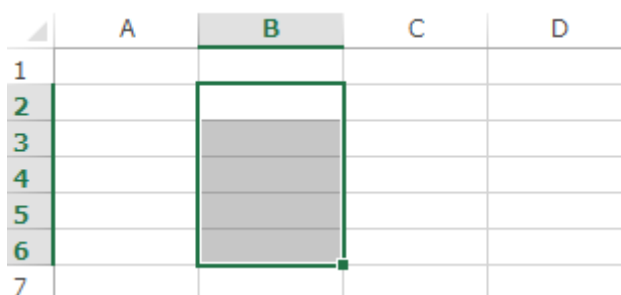
Part. 1 のプログラムはすべて中身が一行だけでした。なので、普通に実行しても、何をどうしたのか、すぐ分かりました。でも、上のようにたくさんの文が並んでいるプログラムの場合、簡単には理解できません。

でも、そここのところを分かりやすく見る方法があります。ステップ実行と言って、一行実行しては、そこでストップし、また一行実行しては、そこでストップ、という方法です。

II. ステップ実行

では、これからステップ実行しながら、マクロの記録機能で記録された VBA プログラムを、一行一行解説してゆきます。が、今はこの解説のぜんぶを分かる必要はありません。ただ、なんとなく「あ、いろいろなオブジェクトがいろいろな仕事をしてるんだ」と思ったなら、それは、あなたがさらに一歩深く、Excel VBA の世界に踏み込んだ証拠です。

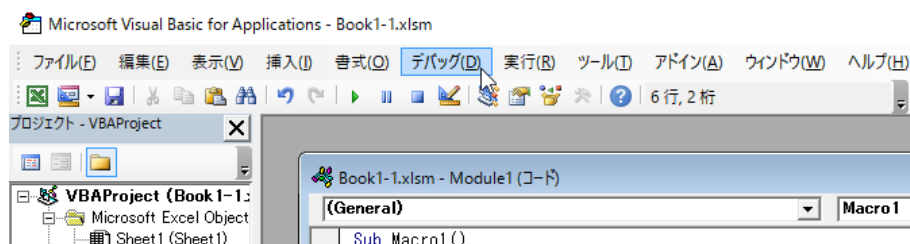
まず、Excel 画面に戻り、シートに入力された値や数式をすべて削除しておきます。



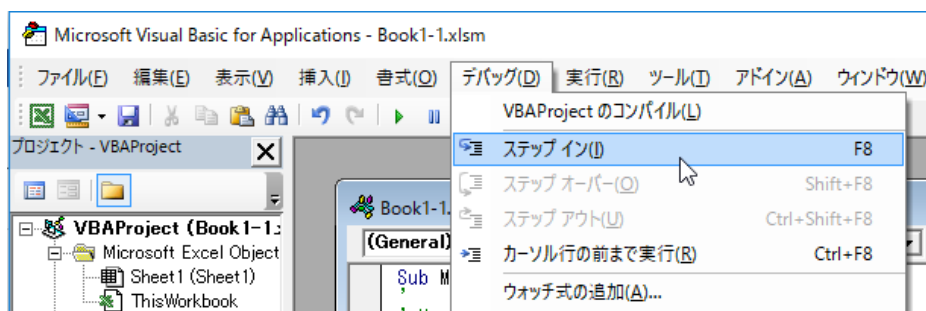
そして、VBE 画面に戻ります。

はい、ここから解説スタート。

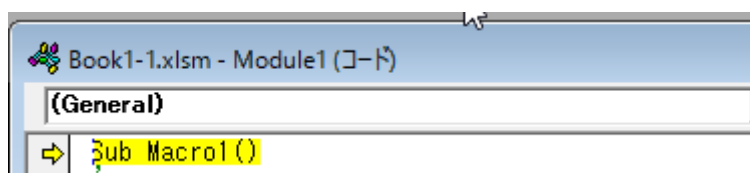
1. メニューバーの[デバッグ] をクリックします。



2. ドロップダウンメニューの中の[ステップイン]をクリックします。



※ 1 と 2 の操作の代わりに、キーボードの[F8]を押してもかまいません。



一行目の Sub Macro1()のバックグラウンドが黄色くなります。これは、「この行に注目してください」という意味です。IT 業界でよく使われるカッコよさげな用語で、「フォーカスする」とも言います。ここでは「これか

ら、この行を実行しますよ」という印になります。

ちなみに、この行の意味は分かりますね。Macro1 という名前の Sub プロシージャはここからですよ、という記述です。なので、この行が実行されても、何も起きません。

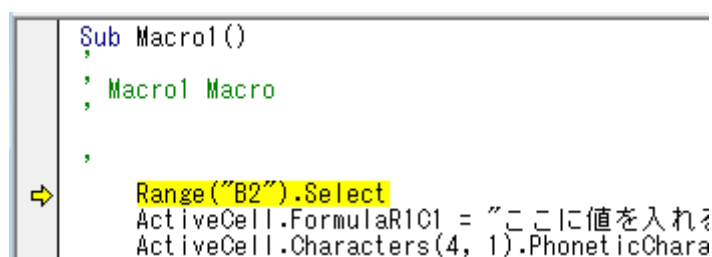
ところで、次の行から、緑色の文字の行がありますね。行頭にシングルクォートが入っています。

 'Macro1 Macro

これはコメントと呼ばれる行です。コメントはプログラムの動きには影響を与えません。オブジェクトに仕事をしてもらうための文ではないのです。では、何のためにあるのかというと、プログラムを読む人のために書く、注釈です。VBA は Excel に指示するための手順書ですから、Excel にとっては分かりやすくても、人にとっては分かりやすくありません。そこを補うために、コメントというものがあるのです。

では次。

3. ここで、1 と 2 の操作を繰り返す ([ステップ イン]をクリックする) か、[F8] キーを押します ([F8]キーを押すと一発なので、こっちがおすすめ)。



```
Sub Macro1()  
' Macro1 Macro  
'  
⇒ Range("B2").Select  
ActiveCell.FormulaR1C1 = "ここに値を入れろ"  
ActiveCell.Characters(4, 1).PhoneticChara
```

すると、黄色いバックグラウンドが Range("B2").Select に移動します。当たり前と言えば当たり前なのですが、VBA プログラムは基本的に上から順に実行されます。順次実行ですからね。

さて、やっとおなじみのオブジェクトが出てきました。Range("B2")は B2 セルのことでしたね。

ここでは、このオブジェクトの "Select" というメソッドが呼び出されています。これは名前の通り、「選択する」という機能を持っています。つまり、このメソッドが実行されたことで、B2 セルが選択されている状態になります。

今はまだ、まだこの行が実行されていないので、Excel シートのほうには、変化がありません。念のため Excel 画面に戻って(やり方は分かりますね?)、見てみましょう。

	A	B	C	D
1				
2				
3				
4				
5				
6				
7				

はい、こんな感じ。

なので、3 と同じ操作をして、先に進めましょう。

- 3 と同様に、マウスで[ステップ イン]をクリックするか、[F8]キーを押して、ステップ インを行う。

```

,
Range("B2").Select
ActiveCell.FormulaR1C1 = "ここに値を入れる"
ActiveCell.Characters(4, 1).PhoneticCharacters = "アタ
ActiveCell.Characters(6, 1).PhoneticCharacters = "イ"

```

黄色いバックグラウンドが次の行に移動しました（カッコよさげな言葉では、「フォーカスが次の行に移った」と言います）。これは「次に実行されるのは、この行です」という意味でしたね。ということは、その前の「Range("B2").Select」の行は実行されたということです。

```

,
Range("B2").Select
ActiveCell.FormulaR1C1 = "ここに値を入れる"
ActiveCell.Characters(4, 1).PhoneticCharacters = "アタ
ActiveCell.Characters(6, 1).PhoneticCharacters = "イ"

```

ここで実行された行

※ Range("B2").Select を囲んでいる赤枠と「ここで実行された行」という注釈は、実際には VBE 画面には現れません。

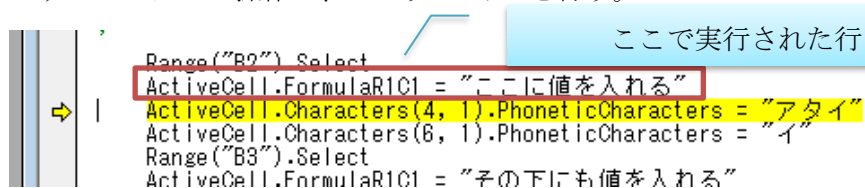
では、Excel 画面に戻って、結果を見てみましょう。

	A	B	C	D	E
1					
2					
3					

確かに、B2 セルが選択された状態に変わっています。

では次へ。

5. マウスまたはキー操作で、ステップ インを行う。



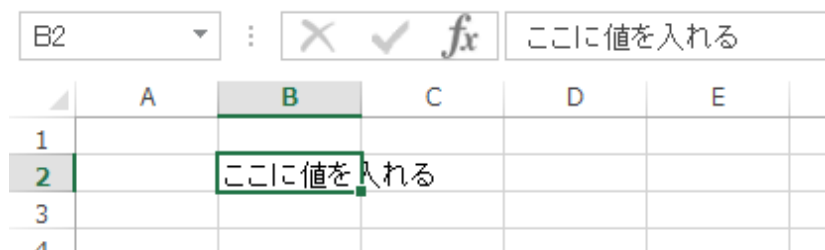
「ActiveCell.FormulaR1C1 = "ここに値を入れる"」

が実行されたはずです。「=」は代入を表すのでしたね。つまり左側に記されたオブジェクトのプロパティに、"ここに値を入れる"という文字列を代入しているところです。でも、この左側（ActiveCell オブジェクトの FormulaR1C1 プロパティ）って、何？

まず ActiveCell というのは現在アクティブなセルを表します。前の前の行で B2 セルの Select メソッドが実行されたことによって、B2 セルが Active になりました。Active とは、キーボードからの入力を受け付けたり、マウス操作でこのセルの何か（たとえば書式）を変更するダイアログを開いたりすることができる状態です。

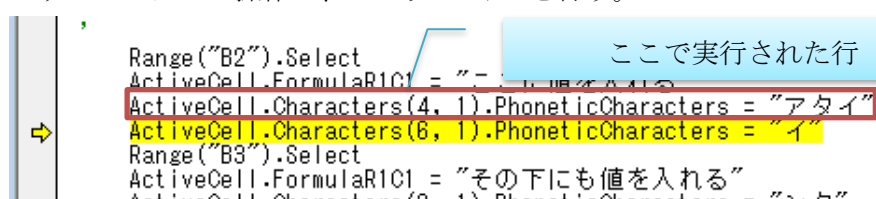
次に FormulaR1C1 というプロパティ。これはセルの数式を意味します。詳しい説明は省略しますが、今まで使ってきた値(Value)というプロパティと異なり、文字列や数値だけでなく、後で出てくるように、数式を入れることができます。

では、Excel 画面を確認。



はい、文字列が入力されています。

6. マウスまたはキー操作で、ステップ インを行う。



ここで実行された行は「ActiveCell.Characters(4, 1).PhoneticCharacters = "アタイ"」です。これは「値」の読み仮名ですね。

ActiveCell.Characters(4, 1)は、現在アクティブな B2 セルに入っている文

字列の、「先頭から 4 つ目の 1 文字」を意味します。文字列の先頭から、

「こ」は、先頭から 1 つ目の 1 文字なので、**Characters(1,1)**

「こ」は、先頭から 2 つ目の 1 文字なので、**Characters(2,1)**

「に」は、先頭から 3 つ目の 1 文字なので、**Characters(3,1)**

「値」は、先頭から 4 つ目の 1 文字なので、**Characters(4,1)**

となります。この「値」という漢字の読みを”アタイ”としますよ、というのが、この行の意味です。Excel2013 では自動的に読み仮名を設定してくれるんです。ちなみに、もし、「値を」の 2 文字、つまり「先頭から 4 つ目の文字から 2 文字」を指定したければ、**Characters(4,2)**となります。

ところでみなさん、一行の左辺(“=”の左側を左辺・右側を右辺と言います)にドットが二つあることに気づきましたか？

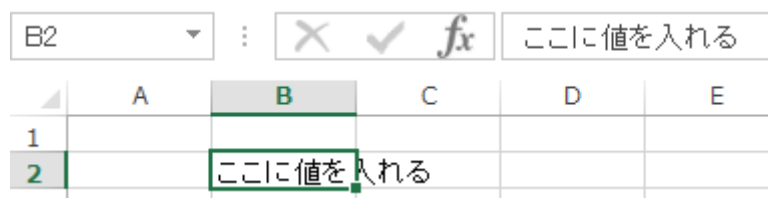
Part.1 で出てきた階層構造ですね。でも、ここではオブジェクトの親子関係ではなく、プロパティの中に、さらにプロパティがあるという親子関係です。「オブジェクト **ActiveCell** のプロパティ **Characters(4,1)** の **PhoneticCharacters** というもう一階層下のプロパティ」という意味です。

こうしたプロパティの階層構造はほかにもあります。たとえばセルのフォントを定義するフォント・プロパティには、大きさを表す”**Sise**”や色を表す”**Color**”など、もう一段下のプロパティがあります。もし、C7 セルのフォントの大きさを 14 にしたければ、次のように書きます。

【例】

Range(“C7”).Font.Size = 14

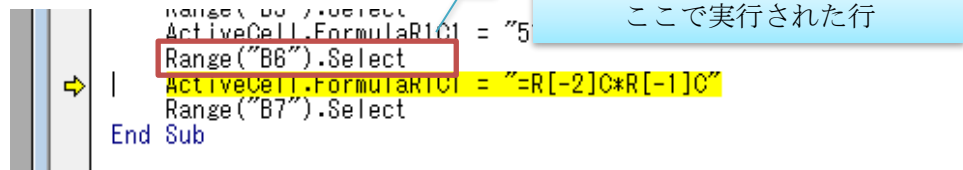
ところで、この **ActiveCell.Characters(4, 1).PhoneticCharacters** プロパティは、Excel の画面に直接表示はされません。したがって、画面は前のステップと変わりません。



さて、この先「**ActiveCell.Characters(6, 1).PhoneticCharacters = "イ"**」から「**Range("B6").Select**」までの 11 行は、これまでに説明した行とほぼ同じなので、特に解説はしません。

「**ActiveCell.FormulaR1C1 = "=R[-2]C*R[-1]C"**」から再開します。

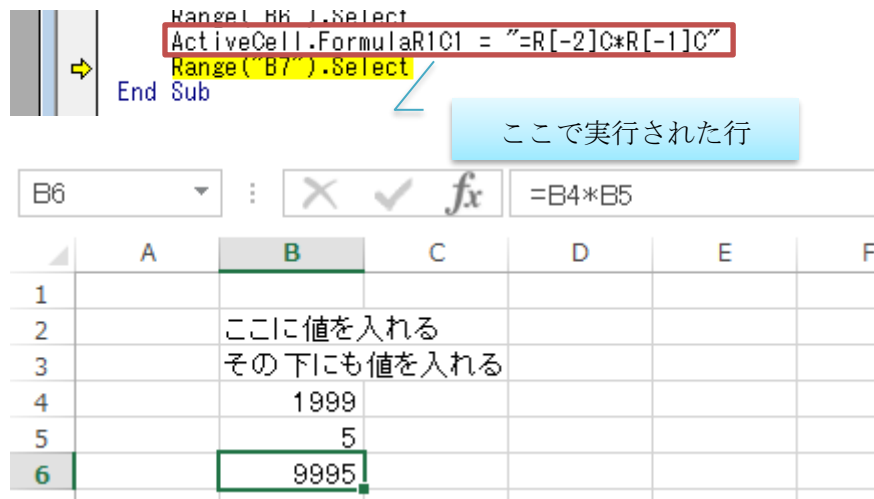
7. マウスまたはキー操作で、ステップ インを行う。



	A	B	C	D	E
1					
2		ここに値を入れる			
3		その下にも値を入れる			
4		1999			
5		5			
6					

前の行で、B6 セルの Select メソッドが動いて、B6 が ActiveCell になっています。

8. マウスまたはキー操作で、ステップ インを行う。



	A	B	C	D	E	F
1						
2		ここに値を入れる				
3		その下にも値を入れる				
4		1999				
5		5				
6		9995				

「ActiveCell.FormulaR1C1 = "=R[-2]C*R[-1]C"」が実行されて、B6 セルに、数式が入っています。Value ではなく、FormulaR1C1 プロパティなので、そういうことができるのです。普通に書くと"=B4*B5"という数式（実際、上の[fx]の右の表示はそうになっていますよね）なのですが、ここでは B4 セルを、現在アクティブな B6 セルからの位置関係で示しています。

- R[-2]は、「アクティブなセルから見て2つ前の行」の意味。つまり 6 行の 2

つ前だから、4 行目。

- Cは「同じ列」という意味。つまり B 列。
というわけで、B4 のセルを指すわけです。

※ 余談ですが、FormulaR1C1 でなく、Formula というプロパティもあって、こちらには、"=B4*B5"が入ります。

Formula bar: $=B4*B5$

	A	B	C	D	E	F
1						
2		ここに値を入れる				
3		その下にも値を入れる				
4		1999				
5		5				
6		9995				

Diagram illustrating the calculation of B6 (9995) based on B4 (1999) and B5 (5). The formula bar shows $=B4*B5$. A callout box explains: "アクティブなセルから見て、2つ前の行・同じ列" (From the active cell, 2 rows up, same column).

ちなみに、もし、D7 であれば、R[1]C[2]になります。

とすると、R[-1]C の意味は分かりますね。一つ前の行の同列、つまり B5 セルです。

で、その2つのセル (B4 と B5) の掛け算を行う、というのが、上の数式の意味です。

9. マウスまたはキー操作で、ステップ インを行う。

```
Range("B6").Select
ActiveCell.FormulaR1C1 = "=R[-2]C*R[-1]C"
Range("B7").Select
End Sub
```

ここで実行された行

B7	:				
	A	B	C	D	E
1					
2		ここに値を入れる			
3		その下にも値を入れる			
4		1999			
5		5			
6		9995			
7					

これも意味は分かりますね。プログラムのには要らない行ですが、マクロを記録する際、B6 セルに数式を入力して Enter をたたいたので、アクティブなセルが B7 に移りました。それがそのまま「Range("B7").Select」というふうに記録されただけです。

10. マウスまたはキー操作で、ステップ インを行う。

はい、完了です。

```
Sub Macro1()  
,  
, Macro1 Macro  
,  
    Range("B2").Select  
    ActiveCell.FormulaR1C1 = "ここに値を入れる"  
    ActiveCell.Characters(4, 1).PhoneticCharacters = "アタイ"  
    ActiveCell.Characters(6, 1).PhoneticCharacters = "イダ"  
    Range("B3").Select  
    ActiveCell.FormulaR1C1 = "その下にも値を入れる"  
    ActiveCell.Characters(3, 1).PhoneticCharacters = "シタ"  
    ActiveCell.Characters(6, 1).PhoneticCharacters = "アタイ"  
    ActiveCell.Characters(8, 1).PhoneticCharacters = "イダ"  
    Range("B4").Select  
    ActiveCell.FormulaR1C1 = "1999"  
    Range("B5").Select  
    ActiveCell.FormulaR1C1 = "5"  
    Range("B6").Select  
    ActiveCell.FormulaR1C1 = "=R[-2]C*R[-1]C"  
    Range("B7").Select  
End Sub
```

VBE の画面から、黄色いバックグラントの行が無くなりました。すべての行を実行し終わったからです。

Excel の画面も前回から変化はありません。

B7	:				
	A	B	C	D	
1					
2		ここに値を入れる			
3		その下にも値を入れる			
4		1999			
5		5			
6		9995			
7					
8					

以上で Macro1 のステップ実行と各行の解説を終わります。

みなさんがこれから自分でプログラムを書いたとき、もしプログラムが自分の思い通りの動きをしなかったら、その間違った動きがどこの行で起きているか、ステップ実行していけば分かりますね？

というわけで、次の章では、みなさんに自分で、中身が複数行あるプログラムを書いてもらいます。

Ⅲ. 複数の命令を順番に実行する

1. 順次実行

「順次実行」という言葉を聞いたことがあるでしょうか？ 読んで字のごとし。順番に実行してゆくことです。順番、という以上、1つだけではありません。二つ以上の手順を実行してゆきます。

多くの手順書は、この順次実行で出来上がっています。例えば。

- ① ファイル・サーバに、管理者権限を持つ ID でログインする
- ② バックアップ・ソフトを起動する
- ③ バックアップ手動実行のジョブを起動する

(To Be Continued…)

運用部門の方は、もっとちゃんとした手順書を使っていることでしょうか、例としてはこんな感じでしょうか。作業を指示された順番に実行してゆくことが求められます。

先ほどまで、みなさんに見てもらっていた「マクロの記録」によって作成された VBA プログラムは、この順次実行のみで出来上がっていました。順次実行はすべてのプログラムの基本です。

2. 記録したマクロを参考に、実際に書いてみる

では、例題を 1 つ。

【例題 5】

次の順番で動く Sub プロシージャを Example05 という名前で書きなさい。

- ① A1 セルに文字列” 45+244” を入れる
- ② B1 セルに数式” =45+244” を入れる
- ③ A2 セルに文字列” 538+12” を入れる
- ④ B2 セルに数式” =538+12” を入れる

【解答】

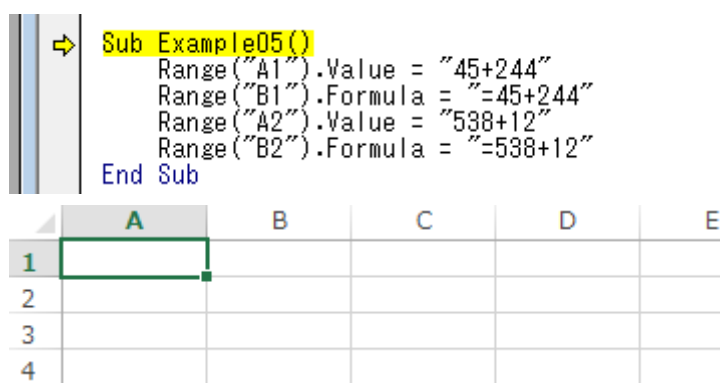
```
Sub Example05()  
    Range("A1").Value = "45+244"  
    Range("B1").Formula = "=45+244"  
    Range("A2").Value = "538+12"  
    Range("B2").Formula = "=538+12"  
End Sub
```

さっきのマクロとは違って、ここではおなじみの Value プロパティと、さっきはチラッとしか出てこなかったけど使いやすい Formula プロパティを使います。

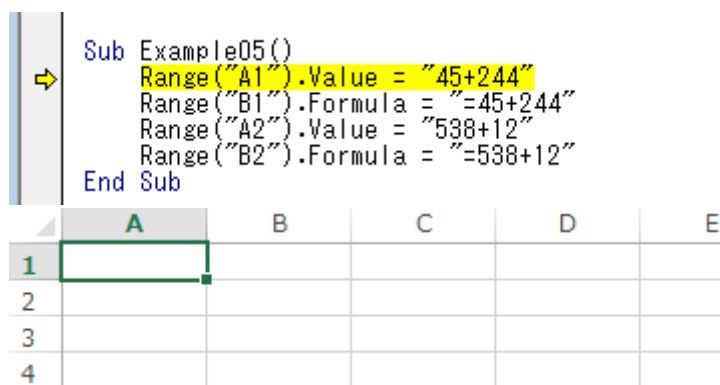
いつものように、解答の通りに標準モジュールに書いてみましょう。そして、書き終わったら、ステップ実行してみましょう。面倒だけど、いちいち Excel 画面に戻って、状態を確認しましょう。実行前に、Sheet1 に入力されているものをいったん全削除して、きれいにすることを忘れないで。

では、1 ステップごとに VBE と Excel の両方のスクリーンショットを撮って、その結果を、解説抜きで、順番に並べます。

1.



2.



3.

```
Sub Example05()
    Range("A1").Value = "45+244"
    Range("B1").Formula = "=45+244"
    Range("A2").Value = "538+12"
    Range("B2").Formula = "=538+12"
End Sub
```

A1 : 45+244

	A	B	C	D	E
1	45+244				
2					

4.

```
Sub Example05()
    Range("A1").Value = "45+244"
    Range("B1").Formula = "=45+244"
    Range("A2").Value = "538+12"
    Range("B2").Formula = "=538+12"
End Sub
```

A1 : 45+244

	A	B	C	D	E
1	45+244	289			
2					

5.

```
Sub Example05()
    Range("A1").Value = "45+244"
    Range("B1").Formula = "=45+244"
    Range("A2").Value = "538+12"
    Range("B2").Formula = "=538+12"
End Sub
```

A1 : 45+244

	A	B	C	D	E
1	45+244	289			
2	538+12				
3					

6.

```
Sub Example05()
    Range("A1").Value = "45+244"
    Range("B1").Formula = "=45+244"
    Range("A2").Value = "538+12"
    Range("B2").Formula = "=538+12"
End Sub
```

⇒

	A	B	C	D	E
1	45+244	289			
2	538+12	550			

7.

```
Sub Example05()
    Range("A1").Value = "45+244"
    Range("B1").Formula = "=45+244"
    Range("A2").Value = "538+12"
    Range("B2").Formula = "=538+12"
End Sub
```

⇒

	A	B	C	D	E
1	45+244	289			
2	538+12	550			

さっきと違って、緑色の枠付きのセル（モノクロ印刷だと色は分かりませんが）が、A1 から移動しないのは、他のセルの Select メソッドを使ってないからです。

では、慣れのために練習問題を 1 つやってみましょう。

【練習問題 7】

次の順番で動く Sub プロシージャを **Exercise07** という名前で書きなさい。書き終わったら、ステップ実行して、動きを確認しなさい。

- ① C1 セルに文字列” 小さいよ” を入れる
- ② C1 セルの文字サイズを 8 にする
- ③ D1 セルに文字列” 大きいよ” を入れる
- ④ D1 セルの文字サイズを 32 にする

ただし、セルの文字サイズを操作する方法については、下記を参照
たとえば、D20 セルの文字サイズを 8 にするには

```
Range("D20").Font.Size = 8
```

D20 セルの文字サイズを 32 にするには

```
Range("D20").Font.Size = 32
```

出来ましたか？ では、例題をもう 1 つ。

【例題 6】

次の順番で動く Sub プロシージャを **Exercise07** という名前で書きなさい。書き終わったら、ステップ実行して、動きを確認しなさい。

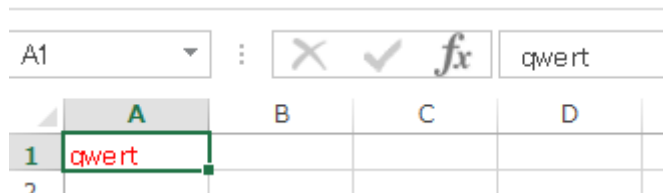
- ① C1 セルに文字列” 赤いよ” を入れる
- ② C1 セルの文字を赤色にする
- ③ D1 セルに文字列” 青いよ” を入れる
- ④ D1 セルの文字を青色にする

まあ、さっきの練習問題とよく似てるので、大丈夫かな。あれ、でも、「ただし、セルの文字色を操作する方法については、下記を参照」がないぞ。

そうです。自分で調べてください。

…といったらグーグル先生に聞いちゃいますか？ もちろん、それも 1 つの方法です。でも、思い出してください。「マクロの記録」機能を使えば Excel 上で自分がやった操作を VBA として記録してくれるんですよね？ ということは、たとえば「セルの文字色を赤にする」とう操作を記録すれば、それがすなわち、「セルの文字色を赤にする」VBA プログラムなんです。つまり、いちいちグーグル先生に聞かなくても、簡単に分かってしまうんです。

というわけで、実際にやってみましょう。



上の図のように、A1 の文字を赤くしました。このとき記録されたマクロは、こうなります。

```

Sub Macro3()
' Macro3 Macro
'
    ActiveCell.FormulaR1C1 = "qwert"
    Range("A1").Select
    With Selection.Font
        .Color = -16776961
        .TintAndShade = 0
    End With
End Sub

```

ふむ、見たことがない書き方ですね。

“With” というキーワードは記述をまとめるのに使います。上の” With” と” End With” に挟まれた行にあるものは、本当は” With” のあとの” Selection.Font” に続くものです。

つまり、With を使ってまとめる前は、

```
Selection.Font.Color = -16776961
```

```
Selection.Font.TintAndShade = 0
```

だったのを、「” Selection.Font” までは同じだから、まとめちゃえ」とやった結果が、With を使った、上の記述になるわけです。

といっても、ルールはあります。「どうせなら” Selection.Font.” と最後にドットまでつけてまとめてしまえ」と思っても、エラーになります。ルール（VBA というプログラム言語のルールですから、「文法」と言います）で、そう決まっているからです。これについては理由を調べるより、「そういうものだ」と覚えてしまったほうが速いでしょう。

いずれにしても、これで文字を赤くするには、二つのプロパティを設定すればよいらしいことが分かりました。

では書いてみましょう。

```

Sub Example06()
    Range("C1").Value = "赤いよ"
    Range("C1").Font.Color = -16776961
    Range("C1").Font.TintAndShade = 0
    Range("D1").Value = "青いよ"
    Range("D1").Font.Color = -4165632
    Range("D1").Font.TintAndShade = 0
End Sub

```

または” With” を使って書くこともできます。

```
Sub Example06()  
    Range("C1").Value = "赤いよ"  
    With Range("C1").Font  
        .Color = -16776961  
        .TintAndShade = 0  
    Range("D1").Value = "青いよ"  
    With Range("D1").Font  
        .Color = -4165632  
        .TintAndShade = 0  
    End With  
End Sub
```

本当のことを言うと、” .TintAndShade = 0” は、なくてもかまいません。これは色の明暗を「-1」から「1」までの数値で指定するプロパティで、0はその中間ですから、いわばデフォルトで、通常はもともとこの値になっています。こんなふうに、余計な文まで記述されてしまうのも、マクロの記録の欠点とされています。

無駄なところを省くと、こんなプログラムになります。

```
Sub Example06()  
    Range("C1").Value = "赤いよ"  
    Range("C1").Font.Color = -16776961  
    Range("D1").Value = "青いよ"  
    Range("D1").Font.Color = -4165632  
End Sub
```

少し説明がめんどくさくなったかもしれませんが。とりあえずは、上記のようなダイエットは後回しでいいと思います。プログラムは動いてナンボですので、余計な部分があってもちゃんと動くものが、余計な部分を削ろうとして動かなくなったプログラムより、よっぽどましなのです。

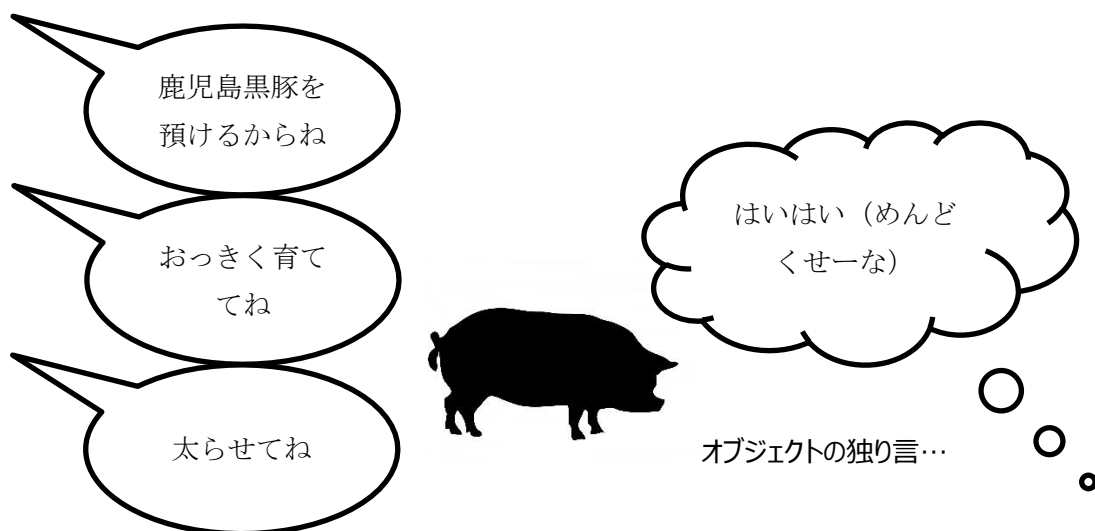
では、最後にもう 1 つだけ練習問題をやって、このパートを終わりにしましょう。

【練習問題 8】

次の順番で動く Sub プロシージャを Exercise08 という名前で書きなさい。書き終わったら、ステップ実行して、動きを確認しなさい。

- ① C3 セルに文字列” 鹿児島黒豚” を入れる
- ② C3 セルの文字サイズを 32 にする
- ③ C3 セルの文字を太文字(Bold)にする

なお、太文字にする方法は、「マクロの記録」を使って調べなさい。



さて、これでこのパートは終わりです。「Excel の裏で、なんかオブジェクトがいろんな仕事をしてるんだ」という感じがしてきましたか？ そして、「VBA プログラムって、書いた命令を上から順に実行してゆくんだね」という流れがつかめたでしょうか？

今は、それで十分です。

Good Job!



次のパートでは、単純に上から実行するのではなく、処理を繰り返してゆく方法を説明します。そのために使わなければならない「変数」という小道具についても、

説明してゆきます。
お楽しみに。

© 2016 ナビオコンピュータ株式会社